arXiv:2609.24021v1 [math.NA] 21 Sep 2026

COST-ACCURACY TRADE-OFFS: NEURAL OPERATOR VS CLASSICAL NUMERICAL SOLVER*

DANIEL ZHENGYU HUANG† AND ANDREW M. STUART‡

Abstract. Neural operators are data-driven models that learn mappings from inputs that parameterize partial differential equations (PDEs), such as spatially varying coefficients, initial conditions, forcing terms, boundary conditions, or geometries, to solution fields or quantities of interest. Once trained, they can serve as surrogates for classical numerical solvers in many-query settings that require repeated evaluations for varying inputs. We address the question of when, and then why, neural operator surrogates outperform classical numerical solvers, in terms of cost for a given accuracy. We focus on the post-training, many-query limit, in which data-acquisition and training costs are treated as fixed and fully amortized. Even in this deliberately favorable regime for neural operators, there are regimes in which classical solvers outperform the surrogate models. We compare the cost-accuracy performance of neural operator surrogates and classical numerical solvers through a reproducible benchmark study comparing neural operators with problem-matched classical solvers on representative problems in computational science and engineering, focusing on prediction error, per-query floating-point cost, and wall-clock runtime. The results reveal no uniformly superior method. Neural operators are most competitive at low-to-moderate accuracy requirements. Their floating-point cost advantage depends strongly on the problem structure, arising when they avoid temporal or nonlinear iterations or predict a reduced quantity of interest rather than a full solution field. Additional wall-clock speedups result from dense tensor operations that are well suited to modern hardware. As the target accuracy is tightened, achieving the required accuracy with neural operators becomes increasingly challenging, and classical solvers outperform surrogates in this regime; thus classical solvers will remain important for verification and high-accuracy computation, even as surrogates emerge as an attractive alternative for parts of the engineering pipeline.

Key words. Surrogate modeling, Neural operator, Parametric partial differential equations, Numerical methods

AMS subject classifications. 65Y20, 68T07, 65N55, 65M70

1. Introduction.

Many computational tasks in science and engineering require repeated evaluations of expensive computational models, often involving parametric partial differential equations. Representative examples include design optimization [69, 118, 81, 35], where many candidate designs must be screened efficiently; uncertainty quantification [119, 16, 33], where probability estimates require a large number of model evaluations; and weather forecasting [66, 7], where predictions must be generated rapidly for evolving physical states, and ensembles of forecasts are often used. In such many-query settings, the dominant computational cost may arise not from a single high-fidelity simulation, but from repeatedly solving a related class of PDE problems for different physical states, parameters, or geometries. This motivates the construction of surrogate models to potentially replace classical numerical solvers.

Classical approaches to surrogate modeling include projection-based reduced-order models [29, 18, 94, 8, 25, 96, 37, 36], Koopman-operator-based methods [101, 100, 86, 87], and Gaussian-process emulators [57, 24, 22]. More recently, neural-

*Submitted to the editors DATE.

Funding: DZH acknowledges funding support from National Key R&D Program of China 2025YFA1018700, and National Natural Science Foundation of China No.62595771. AMS acknowledges funding support from a USA Department of Defense Vannevar Bush Faculty Fellowship (award N00014-22-1-2790). This work was initiated when DZH was a postdoctoral scientist at Caltech, also supported by award N00014-22-1-2790.

†Beijing International Center for Mathematical Research, Center for Machine Learning Research, Peking University, Beijing, China (huangdz@bicmr.pku.edu.cn).

‡California Institute of Technology, Pasadena, CA (astuart@caltech.edu).

network-based surrogates [119, 58, 49] have attracted considerable attention because of their ability to approximate high-dimensional maps and their compatibility with modern accelerator hardware. *Neural operators* [6, 71, 64, 78] extend this perspective by learning mappings between input and output functions rather than between fixed finite dimensional vectors. Representative architectures include DeepONet [78], PCA-Net [49, 6], graph-based neural operators [2, 88], kernel-based neural operators [71, 64, 70, 34, 11, 42], and attention-based neural operators [17, 111, 15]. A central attraction of neural operators is that they formulate the learning problem as a mapping between function spaces and introduce discretization only for numerical computation, thereby providing a degree of robustness across discretizations and resolutions. However, this property alone does not establish a computational advantage over classical numerical solvers.

Training neural operators to a prescribed accuracy may require substantial investments in data generation, model capacity, and offline optimization. We deliberately set these offline costs aside and study the post-training, many-query limit, in which the dataset and trained model are fixed and query-independent costs are fully amortized. Our comparison between neural operator surrogates and classical numerical solvers therefore concerns the cost of answering a new query at comparable accuracy. Even in this deliberately favorable regime, recent studies have shown that some reported inference-efficiency advantages of neural PDE surrogates rely on comparisons with weak baselines or with non-comparable accuracy requirements [83]. Cost-accuracy analysis is therefore essential for understanding the practical value of these methods and guiding their use. For classical numerical solvers, cost-accuracy behavior is relatively well understood through a conjunction of sharp error estimates and computational cost estimates driven, primarily, by the cost of standard linear algebra and optimization routines. For neural operators, by contrast, a comparable understanding remains incomplete, primarily because sharp error estimates are not available. Existing theory primarily addresses approximation properties [21, 78, 64, 63, 68, 107] and statistical convergence rates [28, 74, 98], but does not directly characterize inference-time cost-accuracy trade-offs. The present work provides a systematic numerical study of these cost-accuracy trade-offs, and addresses the following question:

> *In the post-training, many-query limit, under what accuracy requirements and problem structures does neural operator inference achieve a favorable per-query cost-accuracy trade-off relative to a well-designed classical numerical solver?*

We measure per-query computational cost using floating-point work and CPU and GPU wall-clock runtime. Floating-point work characterizes algorithmic complexity, whereas wall-clock runtime also reflects implementation, parallelism, memory access, and hardware utilization.

1.1. Contributions. The main contributions and empirical findings are as follows:

- We establish three benchmarks for evaluating the relative merits of neural operators and classical numerical methods.
- At low-to-moderate accuracy requirements, neural operators can substantially outperform classical numerical solvers, in terms of wall-clock runtime at comparable error levels, in all three benchmarks.
- At higher accuracy requirements, the tested neural operators are not cost-competitive with the classical numerical solvers in all three benchmarks.

More specifically, we formulate a post-training, many-query framework and apply

it to three numerical experiments: 2D Darcy flow, 2D time-dependent incompressible Navier-Stokes flow, and 3D vehicle aerodynamics. In the first two experiments the output is the entire solution field; in the vehicle case, the output is the surface pressure coefficient. The corresponding classical baselines are geometric multigrid, a Fourier pseudospectral solver, and an `OpenFOAM`-based finite volume solver. Across all three benchmarks, we use a modified Fourier neural operator architecture, enabling us to better isolate the effects of problem structure from those of architectural choice. The code, data generation procedures, solver baselines, neural operator architectures, and training protocols are available at

https://github.com/Zhengyu-Huang/Cost-accuracy-trade-off,

complementing earlier reproducible benchmark and software efforts [3, 80, 120, 20].

The observed online advantages arise through different computational mechanisms. For Darcy flow, the principal runtime benefit arises from hardware-efficient dense tensor operations rather than reduced floating-point work. For Navier-Stokes flow, inference reduces sequential depth by replacing many time steps with learned updates. For vehicle aerodynamics, direct prediction of the surface quantity of interest avoids a full volumetric solution and nonlinear iteration. These results show that the many-query limit alone does not guarantee an advantage; the outcome depends on which components of the classical computation are bypassed by the surrogate neural operator.

The experiments also expose limitations that must be addressed before reliable deployment. For the tested neural operators, additional training data or model capacity yields diminishing improvements and may still fail to meet stringent accuracy requirements. In time-dependent problems, errors can accumulate rapidly during long-horizon prediction. More generally, performance may vary substantially across inputs, raising even greater concerns for out-of-distribution inputs; ignoring offline training costs, as we have done here, is only reasonable if the trained model generalizes well. Reliable use in many-query engineering workflows will therefore require uncertainty quantification [120, 106, 110], methods for detecting inaccurate predictions, and hybrid strategies [115] that use data assimilation [99] and classical solvers to verify, correct, or replace neural predictions when higher accuracy and reliability are required.

1.2. Organization. The remainder of the paper is organized as follows. In Section 2, we define the many-query problem and the cost-accuracy comparison scope. In Section 3, we outline the experimental setup, including the benchmarks, classical solvers, and neural operators. Section 4 presents the numerical results, the corresponding cost-accuracy trade-offs, and underlying acceleration mechanisms and limitations of neural operators. We conclude in Section 5.

2. Problem formulation. In this section, we formulate the many-query PDE task and the comparison framework used throughout the paper. We first specify the parametric PDE and the requested output, and then introduce cost models for both classical solvers and neural operators. Finally, we consider the many-query limit and study the cost-accuracy trade-offs between neural operator surrogates and classical numerical solvers. We measure accuracy using a norm on the output space and computational cost in terms of floating-point work or wall-clock time.

2.1. Many-query problem. Consider a parametric PDE together with an associated output map:

$$\mathcal{R}_D(u; a) = 0, \qquad u_{\mathrm{q}} = \Phi_D(u). \tag{2.1}$$

Here, $D \subseteq \mathbb{R}^d$ denotes the geometric configuration and determines a physical PDE domain Ω_D, on which the PDE solution u is defined. The input a collects the query-dependent problem data, which may, for example, be specified throughout Ω_D or on part of its boundary. The requested output u_{q} may likewise be defined throughout Ω_D, on part of its boundary, or as a finite dimensional vector. Examples of input a include spatially varying coefficients, initial conditions, forcing terms, boundary conditions, or other problem parameters. When the geometry varies, D is also part of the query. The output map Φ_D extracts a prescribed quantity of interest and may simply be the identity map.

Solving (2.1) and evaluating Φ_D defines the input-output map

$$\mathcal{G}^\dagger : (a, D) \longmapsto u_{\mathrm{q}}. \tag{2.2}$$

In the benchmarks considered here, we assume that, after any problem-specific encoding, both the input a and output u_{q} are represented as functions on the same domain D. For the full-field benchmarks, $D = \Omega_D$. We define the input and output spaces by

$$\mathcal{A} = \bigsqcup_{D \in \mathfrak{D}} \{a : D \to \mathbb{R}^{d_a}\} \times \{D\}, \qquad \mathcal{U} = \bigsqcup_{D \in \mathfrak{D}} \{u_{\mathrm{q}} : D \to \mathbb{R}^{d_u}\},$$

where $\mathfrak{D}$ is a family of admissible geometric configurations. Let μ be a probability measure on $\mathcal{A}$. The input-output pairs then have the joint law $\left(\mathrm{Id}, \mathcal{G}^\dagger\right)^{\#}\mu$ on $\mathcal{A} \times \mathcal{U}$. The goal of surrogate modeling is to learn an efficient approximation to the target operator $\mathcal{G}^\dagger$ from samples drawn from this law.

2.2. Cost-accuracy model. Suppose that a computational workflow must answer N_{q} queries drawn from μ over its deployment lifetime, with an error tolerance ε. The following cost model applies to both floating-point work and wall-clock runtime.

For each query, a classical numerical method discretizes and solves the governing equations and then applies Φ_D. Its lifecycle cost is modeled as

$$C^{\mathrm{life}}_{\mathrm{solver}}(N_{\mathrm{q}}, \varepsilon) = N_{\mathrm{q}} C_{\mathrm{solver}}(\varepsilon), \tag{2.3}$$

where C_{solver} is the mean, under μ, cost per query required to attain the prescribed accuracy ε.

A neural operator workflow additionally incurs offline costs for data acquisition and training. We decompose the lifecycle cost as

$$C^{\mathrm{life}}_{\mathrm{NO}}(N_{\mathrm{q}}, \varepsilon, N) = C_{\mathrm{data}}(N, \varepsilon) + C^{\mathrm{train}}_{\mathrm{NO}}(N, \varepsilon) + N_{\mathrm{q}} C^{\mathrm{infer}}_{\mathrm{NO}}(\varepsilon), \tag{2.4}$$

where C_{data} is the cost of acquiring N training pairs, $C^{\mathrm{train}}_{\mathrm{NO}}$ is the optimization cost, and $C^{\mathrm{infer}}_{\mathrm{NO}}$ is the mean cost of one inference. If all training outputs are newly generated by the classical numerical solver at tolerance ε, then $C_{\mathrm{data}}(N, \varepsilon) \approx N C_{\mathrm{solver}}(\varepsilon)$. Existing or shared data lead to a different offline cost.

Multifidelity [95] and hybrid [115, 51] strategies are also used to further improve the lifecycle cost-accuracy trade-off for neural operator surrogates. However, we focus on a direct comparison between standalone classical solvers and standalone neural operators; coupled approaches are left for future work.

2.3. Cost-accuracy comparison in the many-query limit. It is useful to distinguish two deployment settings. In a data-scarce setting, a surrogate may be constructed to answer a finite number of queries, in which case data acquisition and

training remain important components of the lifecycle cost. By contrast, a task-specific surrogate trained on a large dataset may be reused persistently for a much larger number of downstream queries, as in a digital wind tunnel used repeatedly for design screening, optimization, and uncertainty quantification. We focus on the idealized limit of the latter setting, $N_{\mathrm{q}} \to \infty$.

For fixed N and ε, the per-query cost of the classical numerical solver is $C_{\mathrm{solver}}(\varepsilon)$. In contrast, the neural operator per-query cost is

$$\begin{aligned}\frac{C_{\mathrm{NO}}^{\mathrm{life}}(N_{\mathrm{q}}, \varepsilon, N)}{N_{\mathrm{q}}} &= \frac{C_{\mathrm{data}}(N, \varepsilon) + C_{\mathrm{NO}}^{\mathrm{train}}(N, \varepsilon)}{N_{\mathrm{q}}} + C_{\mathrm{NO}}^{\mathrm{infer}}(\varepsilon) \\ &\to C_{\mathrm{NO}}^{\mathrm{infer}}(\varepsilon) \quad \text{as } N_{\mathrm{q}} \to \infty.\end{aligned}$$

Thus, the per-query contribution of the fixed offline costs tends to zero. This limit deliberately favors the neural-operator workflow by allowing data acquisition and training to be fully amortized.

Therefore, in the many-query limit, neural operator inference has a cost-accuracy advantage at tolerance ε, when

$$\lim_{N_{\mathrm{q}} \to \infty} \frac{C_{\mathrm{NO}}^{\mathrm{life}}(N_{\mathrm{q}}, \varepsilon, N)}{C_{\mathrm{solver}}^{\mathrm{life}}(N_{\mathrm{q}}, \varepsilon)} < 1,$$

which is equivalent to

$$C_{\mathrm{NO}}^{\mathrm{infer}}(\varepsilon) < C_{\mathrm{solver}}(\varepsilon). \tag{2.5}$$

The present work therefore asks whether (2.5) holds, in which accuracy regimes, and through which computational mechanisms.

3. Experimental setup. To test the many-query criterion in (2.5), we compare problem-matched classical solvers with the same kernel-based neural operator family on three PDE benchmarks. For each method, we vary the numerical or architectural configuration and report every resulting error-cost pair. Floating-point cost and wall-clock runtime cost, versus error, are reported separately.

3.1. Three benchmarks and problem-matched baselines. We consider three benchmarks motivated by subsurface geophysics, geophysical flow forecasting, and aerodynamic design. They are not intended to survey all classes of parametric PDEs. Rather, they span a static elliptic problem, a time-dependent periodic flow, and a geometry-dependent turbulent-flow problem, and are selected to explore three potential sources of online advantage for neural operators: hardware-efficient execution, reduction of sequential time-stepping depth, and task reduction through direct prediction of a partial observable. In each case, we compare the neural operator with a baseline defined by a problem-matched classical solver. Table 1 summarizes the setup.

The Darcy benchmark tests whether neural operator inference can achieve a lower matched-accuracy cost than geometric multigrid for a coercive elliptic problem, for which the classical algorithm has nearly optimal complexity. The baseline is implemented using `Firedrake` and `PETSc` [97, 5]. The Navier-Stokes benchmark tests whether a learned time-advance map can outperform an accurate pseudospectral solver with RK4 time integration by reducing sequential time-stepping depth. The baseline is implemented in `GeophysicalFlows.jl` [26]. The vehicle benchmark tests whether direct prediction of the surface pressure coefficient can reduce online

Benchmark and application	Requested output and error	Classical baseline	Acceleration mechanism
Darcy flow (subsurface geophysics)	Pressure field; relative L^2 error	finite elements with geometric multigrid	Hardware utilization
Incompressible Navier-Stokes flow (geophysical flow forecasting)	Future vorticity field; relative L^2 error	Fourier pseudospectral discretization with RK4 time integration	Reduction of sequential time-stepping depth
Vehicle aerodynamics (digital wind tunnel)	Surface pressure coefficient; relative L^1 error	Finite-volume RANS discretization with SIMPLE iteration	Output reduction and avoidance of nonlinear iterations

TABLE 1

The three benchmarks, motivating applications, requested outputs, problem-matched classical baselines, and potential sources of online advantage for neural operators.

cost by avoiding both a full volumetric solution and the nonlinear iterations of a finite-volume RANS solver. The baseline uses `OpenFOAM` [108, 104], a representative industrial computational fluid dynamics (CFD) solver.

3.2. Kernel-based neural operator. Kernel-based neural operators approximate mappings between function spaces using layers that combine learned kernel integral operators, pointwise transformations, and nonlinear activations [2, 64, 67]. The kernels may be localized in physical space [2], parameterized using Fourier modes [71], multiwavelets [38], or Laplace-Beltrami eigenbases [9, 19, 103], or made input-dependent through attention mechanisms [17, 15]. Across all three benchmarks, we use a modified Fourier neural operator (FNO) [71] designed to accommodate problems with variable geometry [73, 114] and to better capture local effects [77, 114, 41]. Using the same architecture across the benchmarks helps distinguish the influence of problem structure from that of architecture choice. Although alternative neural operator architectures may yield different computational costs and attainable accuracies, the acceleration mechanisms identified here are shared by many such architectures, and our conclusions are therefore expected to remain relevant more broadly. Broader architecture comparisons are available in [79, 50], and an additional comparison with a standard FNO is provided in the supplement in subsection SM3.1.

Given an input pair (a, D), the model first lifts a to a latent field, applies L kernel integral operator layers, and then projects the final latent field to the requested output:

$$\begin{aligned} g_0 &= \mathcal{P}(a), \\ g_i &= \mathcal{L}_i^D g_{i-1} = g_{i-1} + \sigma\big(\mathcal{K}^D_{\mathrm{long},i} g_{i-1} + \mathcal{K}^D_{\mathrm{short},i} g_{i-1}\big), \qquad i = 1, \dots, L, \\ \mathcal{G}(a, D; \theta) &= \mathcal{Q}(g_L). \end{aligned} \tag{3.1}$$

Here $\mathcal{P}$ and $\mathcal{Q}$ are pointwise lifting and projection maps, respectively, and each latent field $g_i : D \to \mathbb{R}^{d_g}$ has d_g channels. Each layer $\mathcal{L}_i^D$ combines a global component $\mathcal{K}^D_{\mathrm{long},i}$, which communicates information across the domain, with a local component $\mathcal{K}^D_{\mathrm{short},i}$, which captures short-range spatial variation. The activation σ is the pointwise GELU function [46]. The residual formulation is used as it facilitates the optimization of deeper architectures [45, 30, 39].

Global interaction. Suppressing the layer index, the global component is chosen to be a truncated Fourier integral operator [71, 64, 89, 90, 62, 67]:

$$
\begin{aligned}
(\mathcal{K}_{\text{long}} g)(x) &= \sum_{\|k\|_\infty \le k_{\max}} \int_D \kappa_k(x,y) W_k^v \, g(y)\, dy \\
&= \sum_{\|k\|_\infty \le k_{\max}} \phi_k(x)\, W_k^v \int_D \overline{\phi_k(y)}\, g(y)\, dy,
\end{aligned} \tag{3.2}
$$

where $k \in \mathbb{Z}^d$ denotes a Fourier mode, $\kappa_k(x,y) = e^{2\pi i k \cdot \frac{x-y}{l}}$, $\phi_k(x) = e^{2\pi i k \cdot x/l}$, l is a characteristic length scale of the domain, and $W_k^v \in \mathbb{C}^{d_g \times d_g}$ is the learned complex matrix associated with mode k. Only modes satisfying $\|k\|_\infty \le k_{\max}$ are retained. This truncation reduces the number of learnable parameters and yields a low-rank spectral representation of the global operator. When the computational domain D is a fixed periodic box discretized on a structured grid, (3.2) can be evaluated efficiently using the fast Fourier transform (FFT):

$$
\mathcal{K}_{\text{long}} g = \mathcal{F}^{-1}\Big(R \cdot \big(\mathcal{F} g\big)\Big), \qquad R(k) = \begin{cases} W_k^v, & \|k\|_\infty \le k_{\max}, \\ 0, & \text{otherwise.} \end{cases} \tag{3.3}
$$

Here $\mathcal{F}$ and $\mathcal{F}^{-1}$ denote the channelwise Fourier transform and its inverse. Thus, the global component applies learned channel mixing to each retained mode. For the Darcy flow and Navier-Stokes benchmarks, the global component is evaluated using the FFT formulation (3.3). For the vehicle aerodynamics benchmark, D is a variable, unstructured surface, and the integral representation (3.2) is evaluated directly by quadrature.

Local interaction. The local component combines a pointwise affine map with a gradient correction [114, 77]:

$$
\mathcal{K}_{\text{short}} g(x) = W g(x) + b + W^{g'} \texttt{SoftSign}\Big(W^g \text{vec}\big(\nabla_D g(x)\big)\Big). \tag{3.4}
$$

Here $W \in \mathbb{R}^{d_g \times d_g}$, $b \in \mathbb{R}^{d_g}$, $W^g \in \mathbb{R}^{d_g \times d_g d}$, and $W^{g'} \in \mathbb{R}^{d_g \times d_g}$ are learned parameters. The correction supplies local derivative information that complements the truncated global representation. The componentwise map $\texttt{SoftSign}(z) = z/(1+|z|)$ controls large discrete gradients near sharp features. We approximate $\nabla_D g$ by centered differences on structured grids and by local least squares on point clouds; on an embedded surface, the latter approximates the tangential gradient.

Training. The lifting, projection, and layerwise parameters are collected in θ. Given N independent training inputs

$$
(a_j, D_j) \overset{\text{i.i.d.}}{\sim} \mu, \qquad j = 1, \dots, N, \tag{3.5}
$$

and corresponding reference outputs $\mathcal{G}^\dagger(a_j, D_j)$, we define the empirical measure $\mu^N := \frac{1}{N}\sum_{j=1}^N \delta_{(a_j, D_j)}$, and the empirical relative error loss

$$
\begin{aligned}
\mathcal{J}_N(\theta) :=& \mathbb{E}_{(a,D)\sim\mu^N}\Big(\frac{\|\mathcal{G}^\dagger(a,D) - \mathcal{G}(a,D;\theta)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a,D)\|_{\mathcal{U}}}\Big) \\
=& \frac{1}{N}\sum_{j=1}^N \frac{\|\mathcal{G}^\dagger(a_j,D_j) - \mathcal{G}(a_j,D_j;\theta)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a_j,D_j)\|_{\mathcal{U}}}.
\end{aligned} \tag{3.6}
$$

Here $\|\cdot\|_{\mathcal{U}}$ denotes the benchmark-specific L^2 or L^1 norm listed in Table 1, with dependence on D suppressed. For the time-dependent Navier-Stokes benchmark, we adapt the training loss to employ the recurrent rollout relative-error loss defined in (4.9).

3.3. Measuring accuracy and cost. We consider finite sets of classical solver and neural operator configurations and report all resulting cost-error pairs. The configurations used for each benchmark are detailed in section 4. Our accounting of online cost begins after each input has been discretized and converted to the representation required by the corresponding implementation. Accordingly, the reported costs exclude geometric preprocessing and mesh generation. For each configuration, per-query cost is quantified separately by floating-point work and wall-clock runtime.

Floating-point work. We count each multiplication or addition as one flop and ignore memory traffic, communication, and parallelization overhead. For the classical methods, the count includes query-dependent assembly and iterative solution or time integration. For the neural operator, it includes lifting, all operator layers, and output projection. The classical solver flop counts are given with the corresponding benchmarks in section 4; the neural operator flop counts are given below.

Throughout this work, n_e denotes the effective spatial discretization size associated with D. Depending on the discretization, it counts grid points, finite element degrees of freedom, or points in an unstructured point cloud. Let $K = (2k_{\max}+1)^d$ denote the number of retained Fourier modes. For fixed input and output dimensions and retaining the leading terms, the inference count for a configuration with latent width d_g and L layers is

$$C_{\mathrm{NO}}^{\mathrm{infer}}(n_e, K, d_g, L) \approx 8LKd_g^2 + \big(2 + (2d+4)L\big)d_g^2 n_e + \begin{cases} 10Ld_g n_e \log_2 n_e, & \text{structured grid,} \\ 12KLd_g n_e, & \text{unstructured grid.} \end{cases} \tag{3.7}$$

The case-dependent final term accounts for transformations between physical and spectral representations in the global interaction: FFTs on a structured grid (3.3) and separable quadrature (3.2) on an unstructured point cloud. The first term accounts for modewise channel mixing, and the second collects the dominant pointwise operations and gradient corrections. For fixed K, d_g, and L, the structured-grid cost is linear in n_e, with a logarithmic correction; the unstructured-grid cost is linear in $K\,n_e$. Detailed counts and implementation choices are provided in the supplement in section SM1. Discussion of the choice of $k_{\max}$, and hence of K, may be found in [67].

Wall-clock runtime. To reflect practical computational workflows, each classical solver is evaluated in its native software environment using the resource allocation specified in Table 2. All timings are obtained on compute nodes equipped with Intel Xeon Platinum 8358 CPUs and NVIDIA A100 GPUs. CPU and GPU timings are reported separately whenever both implementations are available. Unlike floating-point operation counts, Wall-clock runtime reflects memory access, parallelism, implementation, and hardware utilization in addition to floating-point work. Because resource allocations differ across methods, the reported timings should be interpreted as time-to-solution measurements for the specified implementations, hardware, and resource allocations.

4. Three cost-accuracy regimes. In this section, we use the three benchmarks to examine three regimes in which neural operators may achieve lower online cost at matched accuracy. Each benchmark is organized in the same way: we first introduce

TABLE 2
Software and computational resources used for the wall-clock evaluation of the classical solver baselines.

Benchmark	Darcy flow	Incompressible Navier-Stokes flow	Vehicle aerodynamics
Baseline software	`Firedrake/PETSc` (Python/C)	`GeophysicalFlows.jl` (Julia)	`OpenFOAM` 12 (C++)
Baseline resource allocation	One CPU core	One CPU core or one GPU	4, 32, or 256 CPU cores, with approximately 10^5 cells per core

the problem-matched classical method and its expected cost-accuracy scaling, then compare the measured cost-accuracy curves and identify the mechanism underlying any observed neural operator advantage, which typically occurs for low-to-moderate accuracy demands, and finally discuss the principal limitation of neural operators revealed by the experiment, typically apparent at higher accuracy.

4.1. Elliptic problems with optimal solvers: Darcy flow. Darcy flow is a stringent test for neural operator inference because geometric multigrid has nearly linear complexity for the resulting coercive elliptic problem. The comparison therefore distinguishes a reduction in floating-point work from an advantage in hardware execution.

We consider the two-dimensional Darcy flow problem on the square domain $D = [0,1]^2$:

$$\begin{aligned} -\nabla \cdot (a \nabla u) &= f && \text{in } D, \\ u &= 0 && \text{on } \partial D. \end{aligned} \tag{4.1}$$

The forcing term is fixed as $f \equiv 1$. Our goal is to learn the solution operator

$$\mathcal{G}^\dagger : \left(a, [0,1]^2\right) \mapsto u; \tag{4.2}$$

the domain D is fixed across all samples, is encoded in the neural operator through the coordinate map $x \mapsto x$ and we do not attempt to learn dependence on varying D. The permeability field, illustrated in Figure 1, is discontinuous and is modeled as

$$a(x) = 1 + 9\mathbf{1}_{\{\tilde{a}(x) \geq 0\}},$$

where $\tilde{a}$ is a centered Gaussian random field with covariance operator

$$\mathcal{C} = (-\Delta + \tau^2)^{-\alpha}.$$

Here $-\Delta$ denotes the Laplacian on D, subject to Neumann boundary conditions on the space of spatial-mean zero functions, with $\alpha = 2$ and $\tau = 3$ following [71]. Thus, a takes values in $\{1, 10\}$, giving a higher contrast permeability field than in the original setup of [71].

4.1.1. Classical baseline and expected scaling. We discretize the Darcy flow equation (4.1) using conforming bilinear Q_1 finite elements on a uniform $n \times n$ quadrilateral mesh of $D = [0,1]^2$, with mesh size $h = 1/n$ and $n_e = n^2$ elements. Let V_h denote the corresponding finite element space with homogeneous Dirichlet boundary conditions. The discrete solution $u_h \in V_h$ satisfies

$$\int_D a \nabla u_h \cdot \nabla v_h \, dx = \int_D f v_h \, dx, \qquad \forall v_h \in V_h.$$

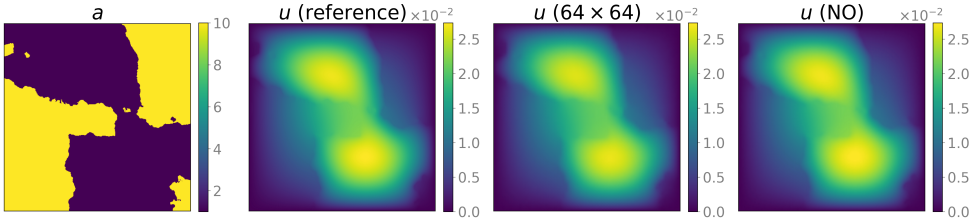

FIG. 1. *Darcy flow for a representative test sample. From left to right: the permeability field, the reference solution, the FEM solution computed with* `Firedrake` *on a coarse* 64×64 *grid, and the neural operator (NO) prediction.*

Both the permeability field a and the source term f are represented by nodal Q_1 functions on the same mesh, and the integrals are evaluated using tensor-product Gaussian quadrature. The resulting discretization yields a sparse symmetric positive definite linear system

$$A_h u_h = b_h.$$

We solve this system using geometric multigrid as a standalone iterative solver, specifically a multiplicative V-cycle [13, 105] on a hierarchy of uniformly refined quadrilateral meshes. On each level, we use Richardson-Jacobi smoothing, while a direct solver is applied on the coarsest grid. For the implementation, we use `Firedrake` [97] as a robust and reproducible finite element baseline, with `PETSc` [5] providing the underlying linear algebra and multigrid infrastructure. The reference data are generated on a 512×512 grid. The linear iterations are continued until the residual reaches machine precision.

The cost of the finite element multigrid solver can be decomposed into an assembly cost and an iterative solve cost. The assembly phase includes the construction of the stiffness matrices, the finest-grid load vector, and the intergrid transfer operators. The solve phase consists of m multigrid V-cycles. Up to lower-order terms, the resulting floating-point cost satisfies

$$C_{\text{solver}} \approx (c_{\text{assemble}} + c_{\text{solve}}\, m)\, n_e. \tag{4.3}$$

The constants c_{assemble} and c_{solve} are both on the order of a few hundred; their detailed values are reported in the supplement in subsection SM2.1. If the coarsest grid is fixed and the number of V-cycles required to reach a fixed relative residual tolerance is independent of the mesh size, so that $m = \mathcal{O}(1)$, then the total solver cost is linear in the number of finest-grid degrees of freedom:

$$C_{\text{solver}} = \mathcal{O}(n_e).$$

Mesh-independent convergence of multigrid under standard smoothing and approximation assumptions is classical; see, for example, [12, 112, 113]. For sufficiently regular solutions, the Q_1 finite element method satisfies the error estimate

$$\|u - u_h\|_{L^2} = \mathcal{O}(h^q), \qquad q \le 2.$$

In the smooth coefficient case, the optimal rate $q = 2$ is recovered. In the present benchmark, however, the permeability field is discontinuous, and the effective convergence rate may therefore be lower [4]. Assuming that the algebraic error is reduced

below the discretization error, achieving an L^2 error of order ε requires $h = \mathcal{O}(\varepsilon^{1/q})$. Since $n_e = \mathcal{O}(h^{-2})$ in two dimensions, the corresponding cost scales as

$$C_{\text{solver}}(\varepsilon) = \mathcal{O}(n_e) = \mathcal{O}(h^{-2}) = \mathcal{O}(\varepsilon^{-2/q}). \tag{4.4}$$

4.1.2. Measured trade-off and acceleration mechanism. For the classical numerical solver, we use the grid sequence $n_e = n \times n$, $n = 16,\ 32, \ldots,\ 512$, with an inexact solve terminated at a relative residual tolerance of 10^{-6}. The neural operator uses the structured-grid implementation with the zero padding strategy for the treatment of nonperiodic boundaries described in the supplement in Theorem SM1.1. The input has $d_a = 3$ channels: the permeability field $a(x)$ and two coordinate functions x_1 and x_2. We fix the training set size at $N = 4000$, determine the number of Fourier modes by setting $k_{\max} = 16$ and set the latent width to $d_g = 64$. We vary the downsampled grid resolution over $n = 32,\ 64$, and 128, and the number of neural operator layers over $L = 4,\ 5,\ 6$, to obtain a range of surrogate implementations.

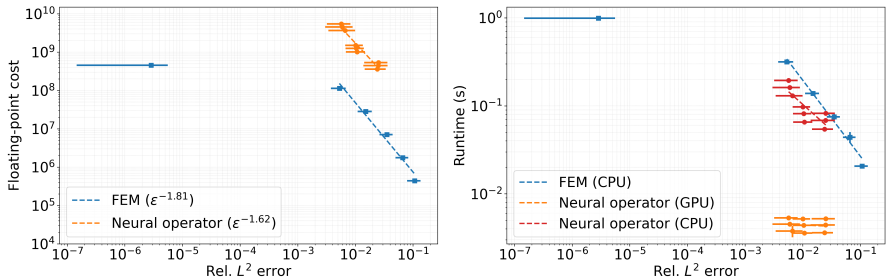

FIG. 2. *Cost-accuracy comparison for Darcy flow between a finite element method with geometric multigrid (FEM) and the neural operator. Left: floating-point operation count versus relative L^2 error. Right: wall-clock runtime in seconds versus relative L^2 error for CPU and GPU implementations. Error bars denote one standard deviation over the test samples. The isolated FEM point at the smallest reported error is obtained on the same $n = 512$ grid as the reference solution but with fewer solver iterations. Because this comparison measures algebraic iteration error but not spatial discretization error, the point is excluded from the FEM power law fit.*

The cost-accuracy comparison is reported in Figure 2. In terms of floating-point operation count, the FEM with geometric multigrid is highly efficient in the high accuracy regime: it typically converges in fewer than 20 multigrid iterations and exhibits nearly optimal scaling, with cost linear in n_e, as shown in (4.3). Its empirical cost-accuracy curve corresponds to $q \approx 1.1$ in (4.4), which is similar to that of the neural operator with $q \approx 1.2$ (leading to the predicted growth of cost as a function of ε in the figure). However the cost-accuracy curve for the FEM method lies far below that of the neural operator, and is far less noisy. The wall-clock comparison leads to a different conclusion: at comparable low-to-moderate accuracy, the neural operator is slightly faster on CPUs and achieves an additional order-of-magnitude speedup on GPUs. This acceleration is largely due to the highly modular structure of the neural operator, whose inference consists primarily of matrix-vector products, FFTs, and pointwise nonlinear activations. These operations can be executed efficiently on modern accelerator hardware. Thus, this benchmark exhibits an implementation- and hardware-execution advantage rather than a reduction in floating-point work.

4.1.3. Limitations and implications. The neural operators in Figure 2 attain relative errors between 5×10^{-3} and 10^{-1}. To examine whether additional data

or model capacity can further reduce this error, we conduct two studies on data downsampled on a 128×128 grid.

First, we fix $k_{\max} = 16$, $L = 6$, and $d_g = 64$, and vary the training-set size N. The corresponding errors are reported in Table 3-top. Error decreases systematically with N, but the improvement slows as the dataset grows. A log-log fit to the displayed values gives an empirical decay of approximately $N^{-0.30}$. Second, we fix $N = 8000$ and compare architectures with different spectral cutoffs $k_{\max}$, widths d_g, and depths L, as shown in Table 3-bottom. Increasing model capacity improves accuracy, but the gains diminish once the error approaches 5×10^{-3}. Published results on related Darcy benchmarks similarly report errors of order 10^{-3} or larger for standalone neural operators [71, 44]. These results indicate an empirical error plateau for the tested configurations and training procedure. Therefore, the tested neural operators do not reach the higher accuracy regime accessible to the finite element solver through mesh refinement.

N	1000	2000	4000	8000
Rel. L^2 test error ($\times 10^{-3}$)	9.18	6.82	5.56	4.91
$(k_{\max}, d_g, L)$	$(4, 16, 4)$	$(8, 32, 5)$	$(16, 64, 6)$	$(32, 128, 7)$
Rel. L^2 test error ($\times 10^{-3}$)	11.37	5.46	4.91	4.70

TABLE 3

Darcy flow: test error versus training set size N for a fixed neural operator architecture $k_{\max} = 16$, $L = 6$, $d_g = 64$ (top); test error versus architecture $(k_{\max}, d_g, L)$ for a fixed training data size $N = 8000$ (bottom).

A useful heuristic decomposition of the in-distribution test error for a neural operator is [78, 50]

$$\mathbb{E}_{(a,D)\sim\mu}\Big(\frac{\|\mathcal{G}^\dagger(a,D) - \mathcal{G}(a,D;\theta_N)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a,D)\|_{\mathcal{U}}}\Big) \approx c_{\text{disc}} h_{\text{eff}}^q + c_{\text{data}} N^{-\beta} + \varepsilon_{\text{opt}}. \tag{4.5}$$

Here θ_N denotes neural operator parameters trained on N samples. The first term represents discretization and architectural representation errors. The effective scale h_{eff} may represent the physical grid spacing when spatial resolution is limiting, or $k_{\max}^{-1}$ when Fourier truncation is dominant. Encoder-decoder truncation produces analogous representation errors. Approximation results that motivate these contributions include [63, Theorem 24] [41, Theorem 2.1] [6, Theorem 3.4][102]. The exponent $q > 0$ is therefore an effective approximation exponent that depends on both the regularity of the target operator and the approximation properties of the architecture. The second term represents the statistical error arising from the approximation of the population loss

$$\mathcal{J}_\infty(\theta) := \mathbb{E}_{(a,D)\sim\mu}\Big(\frac{\|\mathcal{G}^\dagger(a,D) - \mathcal{G}(a,D;\theta)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a,D)\|_{\mathcal{U}}}\Big) \tag{4.6}$$

by the empirical loss (3.6) based on N training samples. The exponent $\beta > 0$ generally depends on the regularity of the target solution operator and on the input distribution; see, for example, [28, Theorem 1.3][98, Corollary 2.8][54, Theorem 4.9][75][74, Theorem 4]. The final term, ε_{opt}, accounts for residual error because training only approximately minimizes the nonconvex empirical objective. This residual may arise from finite optimization time, stochastic gradient noise, or convergence to a suboptimal stationary point [10, 23].

The data and architecture studies show diminishing improvements, suggesting that fixed resolution representation error or residual optimization error may now be limiting. Achieving substantially smaller errors with a stand-alone neural operator therefore becomes increasingly difficult, although recent studies suggest that second-order optimizers can improve attainable accuracy in related scientific machine learning models [60, 56]. For applications requiring stringent accuracy, hybrid methods may offer a more promising path: the neural operator supplies an initial guess, coarse correction, or component of a preconditioner, while a classical solver delivers the final high accuracy solution [115, 51].

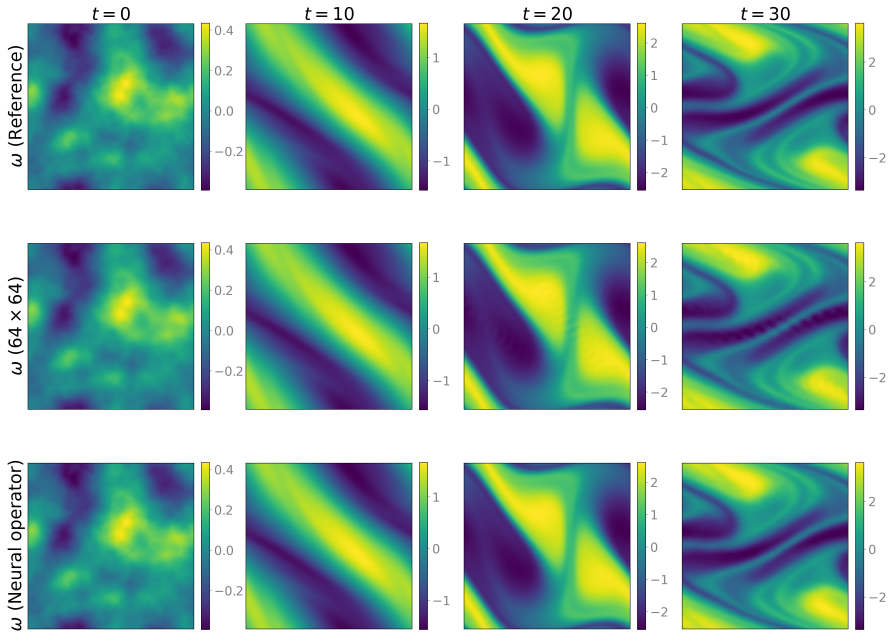

FIG. 3. *Incompressible Navier-Stokes flow for a representative test trajectory at* $t = 0, 10, 20,$ *and* 30 *(columns). From top to bottom: the* 256×256 *reference solution, the spectral solution computed with* `GeophysicalFlows.jl` *on a coarse* 64×64 *grid, and the neural operator rollout prediction.*

4.2. Time-dependent problems: incompressible Navier-Stokes flow. This time-dependent benchmark tests a different mechanism. To advance the solution by one unit of physical time, the explicit pseudospectral solver performs many CFL-limited RK4 steps, whereas the neural operator applies one learned time-advance map. The potential advantage is reduced sequential depth.

We consider the two-dimensional incompressible Navier-Stokes equations on the periodic square domain $[0, 1]^2$; thus, $D = \mathbb{T}^2$, the two-dimensional unit torus. In vorticity form, the equation is

$$\partial_t \omega + v \cdot \nabla \omega = \nu \Delta \omega + f,$$

where ω is the scalar vorticity, ν is the viscosity, and v is the divergence-free velocity recovered from ω through the periodic Biot-Savart law. We consider fixed vorticity forcing $f_{\mathsf{bench}}(x) = 0.1\big(\sin(2\pi(x_1 + x_2)) + \cos(2\pi(x_1 + x_2))\big)$, following [71]. Under this Kolmogorov-type forcing, the flow becomes chaotic after an initial transient; see

Figure 3. The goal is to learn the unit-time solution operator

$$\mathcal{G}^\dagger : \big(\omega(t), f_{\mathsf{bench}}, \mathbb{T}^2\big) \mapsto \omega(t+1).$$

By iterating the learned map, we obtain a recurrent surrogate for predicting flow transitions and certain chaotic features over longer time horizons. In this problem, both the vorticity forcing f and the domain D are fixed across all samples, but they are included as input features to enrich the representation. The initial vorticity ω_0 is modeled as a centered Gaussian random field with covariance operator

$$\mathcal{C} = \tau^{2\alpha-d}(-\Delta + \tau^2)^{-\alpha},$$

where $-\Delta$ denotes the Laplacian on periodic, spatial-mean-zero functions. Following [71], we take $\alpha = 2.5$, $\tau = 7$, $\nu = 10^{-4}$. With this choice, $\omega_0 \in H^{q_0}(D)$ almost surely for every $q_0 < \alpha - d/2 = 1.5$.

4.2.1. Classical baseline and expected scaling. We employ a Fourier pseudospectral discretization in space [48], together with a fourth-order Runge-Kutta method (RK4) in time. The domain is discretized on a uniform periodic $n \times n$ grid with $n_e = n^2$ points and mesh size $h = 1/n$. In this setting, the Fourier coefficients $\widehat{\omega}_k$ of the vorticity satisfy

$$\partial_t \widehat{\omega}_k = -\widehat{(v \cdot \nabla \omega)}_k - \nu |2\pi k|^2 \widehat{\omega}_k + \widehat{f}_k,$$

where k denotes the Fourier index. Spatial derivatives, the Laplacian, and the stream-function Poisson solve used to recover the velocity through the Biot-Savart relation, are diagonal in Fourier space and can therefore be evaluated efficiently using FFTs, whereas the nonlinear advection term is evaluated pseudospectrally. The time step is chosen to satisfy both the advective CFL condition and the explicit viscous stability condition. Compared with lower-order explicit schemes, RK4 typically permits a larger time step for a prescribed accuracy. A comparison with exponential time-differencing schemes [27] is provided in the supplement in subsection SM3.2; the qualitative cost-accuracy conclusions remain unchanged.

For the implementation, we use `GeophysicalFlows.jl` [26], a Julia package for Fourier-based pseudospectral solvers on periodic domains. The package supports both CPU and GPU execution. The reference data are generated on a 256×256 grid, with a sufficiently small time step $\Delta t = 10^{-3}$. For each sampled initial vorticity field ω_0, we simulate the trajectory up to $T = 50$ and store snapshots at integer times.

The cost of the explicit time-dependent solver is the product of the number of time steps n_t and the cost per step. The latter is determined by the four RK4 stages and the cost of evaluating the right-hand side. For the present pseudospectral method, the dominant cost at each stage arises from the FFT-based evaluation of the right-hand side. Up to lower-order terms, the total floating-point operation count satisfies

$$C_{\text{solver}} \approx n_t \left(c_{\text{assemble}}\, n_e \log_2 n_e + c'_{\text{assemble}}\, n_e\right), \tag{4.7}$$

where c_{assemble} and c'_{assemble} denote the leading FFT and non-FFT contributions, respectively. Both constants are on the order of a few hundred, and their detailed values are given in the supplement in subsection SM2.2. At a fixed output time T, the fully discrete error contains spatial and temporal components:

$$\|\omega(T) - \omega_{n,\Delta t}(T)\|_{L^2} = \mathcal{O}\big(h^{q_T} + \Delta t^4\big),$$

where $\omega_{n,\Delta t}(T)$ is computed with mesh size h and time step Δt, and q_T denotes the effective spatial convergence exponent over the resolutions considered. The spatial error reflects both the Fourier pseudospectral discretization and the regularity of the solution. Although the initial vorticity has only finite Sobolev regularity, viscous smoothing increases its regularity at positive times and may yield spectral spatial convergence [32]. The RK4 method contributes the 4th-order temporal error $\mathcal{O}(\Delta t^4)$. In the present setting, the viscosity is small, and the advective CFL condition dominates, so for fixed final time T,

$$\Delta t = \mathcal{O}(h) = \mathcal{O}(n^{-1}), \qquad n_t = \mathcal{O}(n) = \mathcal{O}(n_e^{1/2}).$$

If the observed error behaves as $\varepsilon = \mathcal{O}\big(n_e^{-\frac{\min\{q_T,4\}}{2}}\big)$, the corresponding cost scales as

$$C_{\text{solver}}(\varepsilon) = \mathcal{O}\Big(n_e^{3/2}\log_2 n_e\Big) = \mathcal{O}\Big(\varepsilon^{-3/\min(q_T,4)}\log_2 \varepsilon^{-1}\Big). \tag{4.8}$$

4.2.2. Measured trade-off and acceleration mechanism. For the classical numerical solver, we use the grid sequence $n_e = n \times n$, $n = 32,\ 64, \ldots,\ 256$, with time step $\Delta t = 1/(2n)$. The neural operator uses the structured-grid implementation, adapted to periodic boundary conditions. The input has $d_a = 4$ channels: the current vorticity field $\omega(x)$, the vorticity forcing $f(x)$, and two coordinate functions x_1 and x_2. We train on 200 trajectories up to $T = 50$, sampled at integer times and yielding approximately 10,000 training data pairs. We fix the rollout horizon (4.9) at $s = 2$, $k_{\max} = 16$, and $d_g = 64$, and vary the downsampled grid resolution over $n = 32$, 64, and 128, the number of neural operator layers over $L = 4, 5, 6$, as for the Darcy example.

Following [71], we consider two evaluation settings. For teacher-forced one-step evaluation, denoted by $T = 1$, each reference state $\omega(t), t = 0, \cdots 29$, is supplied as input to predict $\omega(t+1)$. For recurrent prediction, denoted by $T = 30$, only $\omega(0)$ is supplied, and the learned unit-time map is applied recursively through $T = 30$. For each test trajectory, we report the discrete trajectory-averaged relative L^2 error, defined as the average of the relative errors over the 30 predicted states. The present setting is more challenging than that in [71], where the history on $[0, 10]$ is provided and only $t \in (10, 30]$ is predicted. During training, the unit-time update map is unrolled for $s = 2$ steps and optimized using the multistep recurrent loss in (4.9).

The cost-accuracy comparison is reported in Figure 4. In terms of floating-point operation count, the Fourier pseudospectral solver with RK4 is highly efficient. Its empirical cost-accuracy behavior is consistent with (4.8): the fitted rates are $q_1 \approx 3.9$ for the teacher-forced one-step prediction and $q_{1:30} \approx 4.0$ for the trajectory-averaged recurrent rollout. Both values are close to the fourth-order temporal-accuracy ceiling predicted by the theory over the resolutions tested. In both evaluation settings, the neural operator replaces the CFL-restricted RK4 steps over each unit-time interval with a single learned update. However, this reduction in sequential depth does not yield a clear advantage in floating-point work at comparable error levels, except possibly near $\varepsilon = 2 \times 10^{-2}$. Although each pseudospectral stage and each learned update has quasilinear spatial cost, the spectral solver additionally requires $O(n_e^{1/2})$ time steps per unit-time interval, whereas the neural operator has a large per-update prefactor. The measured wall-clock runtime nevertheless remains favorable to neural inference on both the CPU and GPU implementations, with speedups of approximately one order of magnitude. Although both methods rely on FFTs and pointwise operations, the classical implementation advances sequential time steps that cannot

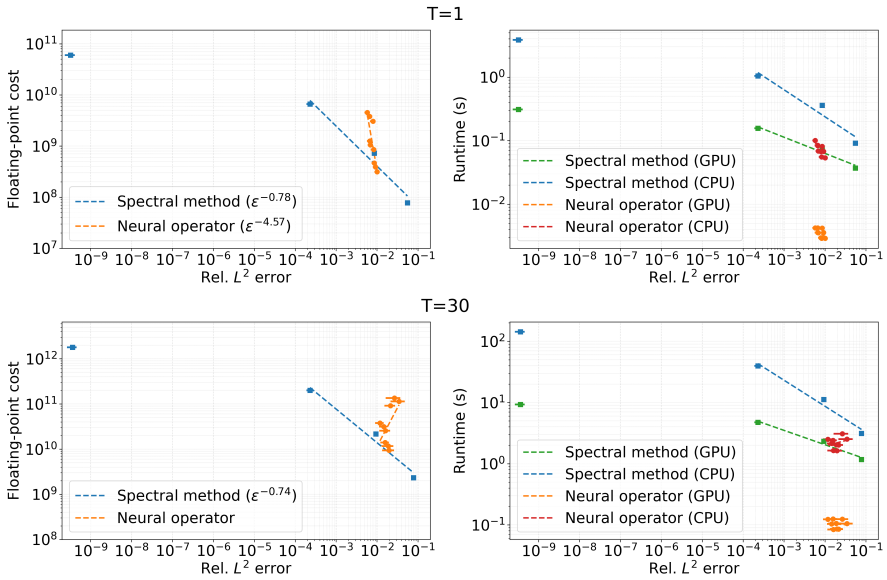

FIG. 4. *Cost-accuracy comparison for incompressible Navier-Stokes flow between the Fourier pseudospectral solver and the neural operator. Top: teacher-forced one-step evaluation* ($T = 1$)*, with each reference state* $\omega(t)$*,* $t = 0, \dots, 29$*, used to predict* $\omega(t+1)$*. Bottom: recurrent rollout from* $t = 0$ *through* $T = 30$*. Left: floating-point operation count versus relative* L^2 *error. Right: wall-clock runtime in seconds versus relative* L^2 *error for CPU and GPU implementations. Error bars denote one standard deviation over the test samples. The isolated smallest error result for the Fourier pseudospectral solver uses the reference grid* ($n = 256$) *with a larger time step. Because it primarily measures temporal error rather than the fully discrete error, it is excluded from the power-law fit.*

be parallelized across time, while the neural operator uses one learned update per unit time. The principal runtime mechanism is therefore reduced sequential depth.

4.2.3. Limitations and implications. Low one-step error does not guarantee an accurate long-horizon rollout. In the 30-step experiment Figure 4, the neural operators attain an average relative L^2 error of order 10^{-2}, but errors accumulate under recurrent application. This growth can be mitigated by using an incremental formulation and training the update map recurrently over s steps with the multistep recurrent loss [116]:

$$\mathcal{J}_s(\theta) := \mathbb{E}_{\substack{\omega_0 \sim \mu, \\ r \sim \mathrm{Unif}\{0,\dots,50-s\}}} \left[\frac{1}{s} \sum_{i=1}^{s} \frac{\|\omega_{r+i} - \mathcal{G}^{(i)}(\omega_r, f, D; \theta)\|_{L^2}}{\|\omega_{r+i}\|_{L^2}} \right], \tag{4.9}$$

where $\mathcal{G}^{(i)}$ denotes the i-step recurrent application of the learned operator. Increasing s generally improves rollout accuracy, but also increases the training cost and memory requirements. To demonstrate the effect of recurrent training, we fix the architecture at $k_{\max} = 16$, $L = 6$, and $d_g = 64$, and train on data downsampled to a 128×128 grid. We compare training horizons $s = 1,\ 2,\ 3$, and include the spectral solver at several grid resolutions as a reference. As shown in Figure 5, the rollout error initially grows approximately linearly for all three horizons. Increasing s improves accuracy but does not eliminate accumulation; one possible explanation for later rapid growth is that recurrently predicted states drift away from those represented in training. In this

regime, one step error is no longer a reliable indicator of rollout accuracy. In contrast, over the finite horizons considered here, the error of the classical solver can be reduced systematically by refining the time step and spatial grid. Although numerical errors also accumulate over time, refinement provides a controlled route to greater accuracy. For neural operators, long horizon recurrent prediction remains challenging, because errors accumulate, especially when transient dynamics must be resolved accurately. Promising directions for future work include structure-preserving objectives based on semigroup constraints [72, 20], symplecticity-preserving architectures for Hamiltonian systems [55, 82], data assimilation strategies [99, 53, 43], and direct spatiotemporal prediction in latent spaces [109, 52].

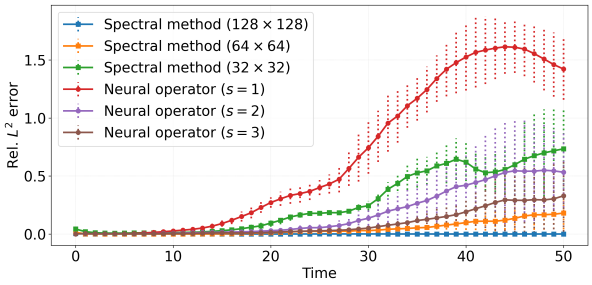

FIG. 5. *Error accumulation over time for incompressible Navier-Stokes flow, comparing the Fourier pseudospectral solver at different spatial resolutions with neural operator models trained using different training horizons. Error bars denote one standard deviation over the test samples.*

4.3. Partial-observable surrogates: vehicle aerodynamics. The vehicle benchmark combines two acceleration mechanisms. The neural operator is a task-specific surrogate intended for applications such as aerodynamic design. It predicts only the surface pressure coefficient and does not provide the volumetric flow field, thereby bypassing both the computation of the full volumetric flow and the nonlinear iterations required by the classical RANS solver.

We consider steady external aerodynamic flow around a vehicle. Let $B \subset \mathbb{R}^3$ denote the computational bounding box and let D denote the vehicle surface. The corresponding fluid domain Ω_D is the portion of B exterior to the vehicle. The flow is modeled by the steady incompressible Reynolds-averaged Navier-Stokes (RANS) equations

$$\nabla \cdot (v \otimes v) = -\frac{1}{\rho}\nabla p + \nabla \cdot \left[(\nu + \nu_{\text{turb}})\left(\nabla v + \nabla v^T\right)\right], \tag{4.10a}$$

$$\nabla \cdot v = 0, \tag{4.10b}$$

where v is the mean velocity, p is the pressure, ρ is the constant density, ν is the molecular kinematic viscosity, and ν_{turb} is the turbulent eddy viscosity provided by the turbulence model, here taken to be the two-equation k-ω SST model [84, 85]. The vehicle boundary is decomposed into stationary body surfaces, where a no-slip condition is imposed, and wheel surfaces, where rotating wall boundary conditions are applied. On the inlet boundary of B, we prescribe a uniform incoming velocity $v_\infty = 30\ m\ s^{-1}$. The outlet, side, and top boundaries are treated using freestream far-field conditions with reference velocity v_∞ and reference pressure p_∞. The ground plane is modeled as a moving wall with velocity v_∞, consistent with the vehicle-fixed frame of reference.

Our goal is to learn a surrogate map from the vehicle geometry to the surface pressure coefficient on D:

$$\mathcal{G}^\dagger : D \mapsto C_p|_D, \quad \text{where} \quad C_p = \frac{p - p_\infty}{\frac{1}{2}\rho v_\infty^2}. \tag{4.11}$$

The boundary conditions and flow parameters are fixed across samples, so the geometry is the only varying input. This remains a parametric PDE problem, since the computational domain Ω_D varies with the vehicle geometry. This is a partial-surrogate setting: the neural operator predicts only the surface quantity of interest $C_p|_D$, rather than the full three-dimensional flow field or quantities like lift or drag. This choice is motivated by aerodynamic design, where the surface pressure distribution directly determines force and loading information and captures local geometric effects on the flow.

For variable geometries, we use the `DrivAerNet++` dataset [31], which contains 8,000 vehicle designs with varying geometric parameters, including overall vehicle length and roof height, and spanning configurations such as fastback, notchback, and estateback. The geometries exhibit complex aerodynamic features representative of industrial vehicle design; see Figure 6.

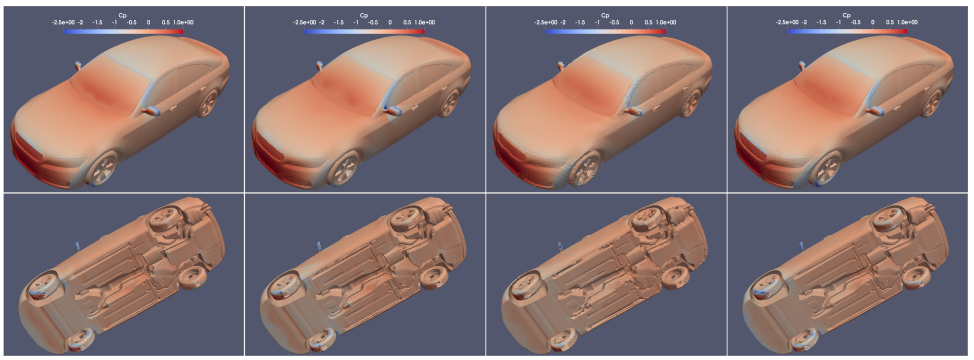

FIG. 6. *Surface pressure coefficient C_p for a representative vehicle from the test set. From left to right: the reference solution obtained using the large mesh and 7000 SIMPLE iterations; `OpenFOAM` predictions obtained using the medium and small meshes; and the neural operator prediction evaluated on a downsampled surface mesh containing approximately* 20,000 *points.*

4.3.1. Classical baseline and expected scaling. We employ a finite volume discretization in space and solve the steady nonlinear RANS system using a segregated SIMPLE fixed-point iteration [93]. The fluid domain is discretized by a cell-centered finite volume method on an unstructured polyhedral mesh, dominated by hexahedral cells, with n_c control volumes. At each outer SIMPLE iteration, the momentum equations are assembled using the current pressure, face fluxes, and turbulent viscosity, and are solved for a predicted velocity using an iterative linear solver with symmetric Gauss-Seidel smoothing. A pressure correction equation is then derived from the discrete continuity constraint and solved using a geometric agglomerated algebraic multigrid method with Gauss-Seidel smoothing. The velocity and face fluxes are subsequently corrected using the updated pressure field. Finally, the two turbulence transport equations are solved as implicit advection-diffusion equations using an iterative linear solver with symmetric Gauss-Seidel smoothing, and the eddy viscosity is updated according to the k-ω SST model. This process is repeated until the residuals

of the velocity, pressure, and turbulence variables satisfy the prescribed convergence criteria.

For the implementation, we use `OpenFOAM` [108, 104], an open-source finite volume CFD package widely used in industrial applications. The setup follows the `drivaerFastback` vehicle benchmark case. The pressure correction equation is solved using `OpenFOAM`'s `GAMG` solver, whereas the momentum and turbulence equations are solved using `smoothSolver`. For each sampled vehicle geometry, the surface mesh is obtained from `DrivAerNet++`, and a geometry-specific volumetric mesh is generated using the `OpenFOAM` meshing pipeline: a background mesh is first created, then locally refined, and finally fitted to the vehicle surface using `snappyHexMesh` [91]. The pressure coefficient fields used to train and evaluate the neural operator are taken from the `DrivAerNet++` dataset [31]. For the classical numerical solver study, we follow the simulation setup described in [31] but independently compute geometry-specific reference solutions using `OpenFOAM` 12 on large volumetric meshes containing more than 20 million cells. Within the class of steady-RANS methods, this benchmark configuration is representative of current high-fidelity industrial practice in automotive aerodynamics. Each reference RANS simulation is advanced for 7000 SIMPLE iterations. Convergence is assessed using the residual histories and the stabilization of the aerodynamic force coefficients over a moving window of 1000 iterations.

The cost of the implicit finite volume solver is the product of the number of SIMPLE iterations n_t and the cost per iteration. Let n_c denote the number of control volumes and n_f the number of internal faces in the finite volume mesh. For shape-regular polyhedral meshes dominated by hexahedral cells, one typically has $n_f \approx 3n_c$. Each SIMPLE iteration consists of assembling or updating the finite volume operators and solving the resulting pressure, velocity, and turbulence model linear systems. Assuming each linear system takes about m iterations of an iterative solver, a rough cost model is

$$C_{\text{solver}} \approx n_t\big(c_{\text{assemble}} n_c + m c_{\text{solve}} n_c\big).$$

The detailed values of c_{assemble} and c_{solve} are on the order of a few thousand and a few hundred, respectively, and are reported in the supplement in SM2.3. The accuracy of the RANS finite volume solver depends not only on discretization error, but also on mesh quality, geometric complexity, linear solver convergence, and fluctuations associated with turbulence modeling. In sufficiently smooth regions and on nearly uniform meshes, second-order finite volume schemes typically yield

$$\|U - U_h\|_{L^1} = \mathcal{O}(h^2),$$

where U and U_h denote the exact and numerical solutions, respectively. On unstructured meshes for complex geometries, however, the effective convergence rate may deteriorate. Assuming an empirical convergence rate

$$\|U - U_h\|_{L^1} = \mathcal{O}(h^q), \qquad q \le 2,$$

and sufficient trace regularity, we assume the relative L^1 error in C_p on the vehicle surface to inherit the same empirical rate. Using $n_c = \mathcal{O}(h^{-3})$ in three dimensions, the cost required to achieve an error of order ε scales as

$$C_{\text{solver}}(\varepsilon) = \mathcal{O}\Big(n_t m \varepsilon^{-3/q}\Big) \tag{4.12}$$

up to the dependence on the SIMPLE and inner linear solver iteration counts.

4.3.2. Measured trade-off and acceleration mechanism. We measure accuracy using the relative L^1 error in the surface pressure coefficient, since integrated pressure loads depend directly on surface integrals. For the classical baseline, we adapt the official `OpenFOAM` 12 `drivaerFastback` workflow [108, 104] to six geometries sampled from `DrivAerNet++` and chosen to cover all vehicle categories. For each geometry, we consider small, medium, and large volumetric meshes and vary the number of SIMPLE iterations. The corresponding 7000-iteration large-mesh solution serves as the numerical reference. For the wall-clock measurements, the CPU allocation is scaled with mesh size to maintain approximately 10^5 cells per core. The reported times are therefore time-to-solution measurements under varying resource allocations, rather than equal-core comparisons. Table 4 reports the configurations and averaged mesh sizes, runtimes, and errors for the `OpenFOAM` predictions over six test geometries. The canonical `drivaerFastback` configuration is reported separately in the supplement in subsection SM3.3 as a representative individual case. The neural operator uses the unstructured-grid implementation. The input comprises the surface coordinates and postprocessed normals, giving $d_a = 6$ channels on n_e points. Each layer incorporates the surface normals through the geometric encoding proposed in [41], motivated by geometry-dependent kernels such as double-layer potentials [47]. We train on $N = 4000$ `DrivAerNet++` samples using a relative L^1 loss. We fix $k_{\max} = 16$, $d_g = 64$, and $L = 4$, and vary the downsampled surface resolution over approximately 10,000, 20,000, and 40,000 points.

Configuration	Large reference	Large	Medium	Small
Mean control volumes $\overline{n_c}$	22,401,336	22,401,336	2,962,931	441,183
SIMPLE iterations n_t	7000	2000	1000	1000
CPU cores	256	256	32	4
Mean runtime (s)	6443.8	1860.1	921.8	547.0
Mean relative L^1 error	Reference	12.06%	18.20%	25.89%

TABLE 4

`OpenFOAM` configurations and results for six selected vehicle geometries drawn from `DrivAerNet++`. Mesh sizes, wall-clock runtimes, and relative L^1 errors in the surface pressure coefficient C_p are averaged over test samples. For each geometry, the error is measured relative to the corresponding geometry-specific reference solution computed on the large mesh using `OpenFOAM` 12 and 7000 SIMPLE iterations.

At approximately 10,000, 20,000, and 40,000 surface points, the average neural operator relative L^1 errors are 13.50%, 12.66%, and 12.03%, respectively. Increasing the surface resolution improves the representation of local geometric and flow features, but the gain is modest. Figure 6 shows that the neural operator model reproduces the dominant pressure pattern for a representative case, qualitatively comparable to the `OpenFOAM` predictions on the medium mesh. For this dataset, neural operator accuracy appears to be limited in part by variability in the RANS-generated training data [111, 1]. In published benchmarks for `DrivAerNet++`, neural operator errors remain of order 10% [31, 111, 1].

The cost-accuracy comparison is shown in Figure 7. The result is consistent with two sources of online advantage. First, the neural operator predicts the requested surface field on n_e points instead of computing a volumetric flow on n_c control volumes, with $n_e \ll n_c$. Second, inference avoids the outer SIMPLE iteration and its associated linear solves. Its floating-point count is consequently below that of the large mesh FVM configuration, although it remains above that of the small mesh configuration.

However, the steeper empirical slope of the neural operator curve suggests that this advantage may diminish in a higher-accuracy regime, even if the neural operator can attain that regime. GPU execution provides an additional wall-clock advantage.

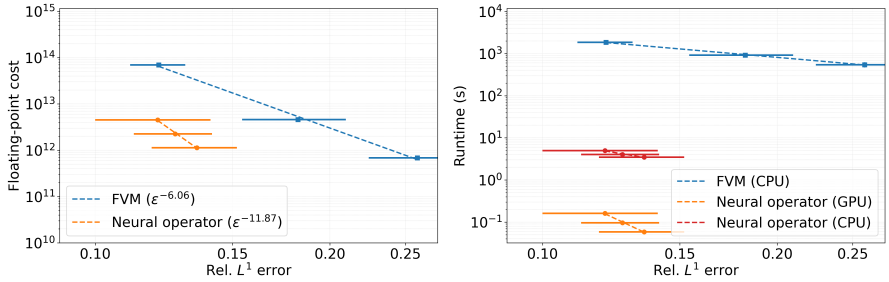

FIG. 7. *Cost-accuracy comparison for vehicle aerodynamics between the finite volume method (FVM) and the neural operator. Left: floating-point operation count versus relative L^1 error. Right: wall-clock runtime in seconds versus relative L^1 error for CPU and GPU implementations. Error bars denote one standard deviation over the test samples.*

4.3.3. Limitations and implications. The principal limitation is reliability across geometries. Although the mean relative L^1 error is moderate, the distribution in Figure 8 has a substantial upper tail. The largest error case corresponds to a relatively tall vehicle with strong pressure variations over the hood. Such prediction failures could compromise downstream engineering tasks, including design optimization. Moreover, because the surrogate predicts only a surface quantity of interest, verification through a full PDE residual is not readily available. Reliable deployment will therefore require uncertainty estimates and out-of-distribution indicators that identify questionable predictions and trigger verification, correction, or replacement by a classical solver.

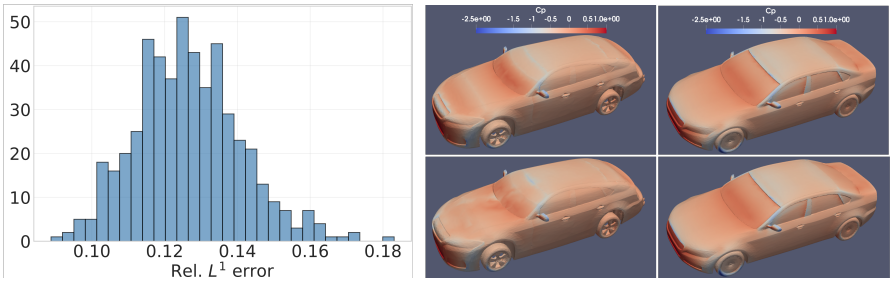

FIG. 8. *Error analysis for vehicle aerodynamics. From left to right: the distribution of test errors, the test case with the largest error, and a representative median error case. For the two visualized cases, the reference solution is shown in the top row and the neural operator prediction in the bottom row.*

5. Conclusion. We have studied when neural operator inference provides a favorable per-query cost-accuracy trade-off relative to strong classical solvers in the post-training, many-query limit. The experiments show that neural operators are competitive when they can attain the requested accuracy and either execute efficiently on the target hardware or bypass a substantial component of the classical computation. Across the cases considered, the observed benefits arise from hardware-efficient tensor operations, reduced sequential or iterative computation, and direct prediction

of task-specific outputs. These mechanisms also explain why no method is uniformly superior: classical solvers reach smaller errors than those attained by the tested neural operators and remain essential for high-accuracy computation and verification.

The many-query limit is motivated by the prospect of scientific models that are reused across increasingly broad classes of problems. Although the neural operators studied here are problem-specific rather than foundation models, the identified acceleration mechanisms indicate how broader reuse could increase their practical value. Realizing this potential will require reliable generalization, indicators of inaccurate or out-of-distribution predictions, and hybrid workflows in which classical solvers verify, correct, or replace neural predictions when higher accuracy or reliability is required.

Acknowledgments. DZH acknowledges the support of the high-performance computing platform of Peking University. After completing the initial draft, DZH used GPT-5.6 and DeepSeek-V3 to obtain suggestions for improving the language and presentation of the manuscript and the documentation of the accompanying software library. The authors reviewed all suggestions and incorporated them as appropriate; they take full responsibility for the final content. The authors also thank George Karniadakis for his helpful suggestions regarding this work.

SUPPLEMENTARY MATERIALS

SM1. Neural Operator. In this section, we give details of the neural operator architecture benchmarked in the present work. It is a modification of the Fourier neural operator [71], designed to better capture local effects [77, 114] and to accommodate problems with variable geometry [73, 114]. The neural operator was introduced and studied in [114, 41] under the names point cloud neural operator (PCNO) and multiscale point cloud neural operator (M-PCNO). We describe the architecture and implementation in subsections SM1.1 and SM1.2, and discuss its general cost-accuracy behavior in subsection SM1.3.

SM1.1. Neural operator architecture. Given an input pair (a, D), with $a : D \to \mathbb{R}^{d_a}$, the neural operator defines a mapping between functions on the domain D

$$\mathcal{G}^\dagger : (a, D) \longmapsto u_{\mathrm{q}}. \tag{SM1.1}$$

It first lifts the input a to a latent feature field $g_0 : D \to \mathbb{R}^{d_g}$, then applies L neural operator layers, and finally projects the resulting feature field onto the desired output space:

$$\begin{aligned} &g_0 = \mathcal{P}(a), \qquad g_i = \mathcal{L}_i^D(g_{i-1}), \quad i = 1, \dots, L, \qquad u_{\mathrm{q}} = \mathcal{Q}(g_L), \\ &\mathcal{G}(a, D; \theta) = \mathcal{Q} \circ \mathcal{L}_L^D \circ \mathcal{L}_{L-1}^D \circ \cdots \circ \mathcal{L}_1^D \circ \mathcal{P}(a). \end{aligned} \tag{SM1.2}$$

Here $\mathcal{P} : \mathbb{R}^{d_a} \to \mathbb{R}^{d_g}$ is a pointwise lifting map, $\mathcal{Q} : \mathbb{R}^{d_g} \to \mathbb{R}^{d_u}$ is a pointwise projection map, and each $\mathcal{L}_i^D$ is a neural operator layer acting on functions defined on D with d_g channels. All hidden layers are taken to have the same channel dimension d_g. The lifting map increases the number of channels from d_a to d_g, thereby embedding the input into a higher dimensional latent space. This higher dimensional representation can capture more complex features and make the underlying relationships more tractable [61, 86, 54].

Suppressing the layer index, each neural operator layer $\mathcal{L}^D$ maps the function space $\{g : D \to \mathbb{R}^{d_g}\}$ into itself. It consists of a linear operator followed by a pointwise nonlinear activation function, both defined at the operator level and independent of the discretization. The linear component is decomposed into a global operator $\mathcal{K}_{\mathrm{long}}$ and a local operator $\mathcal{K}_{\mathrm{short}}$, so that

$$\mathcal{L}^D g = g + \sigma\Big(\mathcal{K}_{\mathrm{long}} g + \mathcal{K}_{\mathrm{short}} g\Big). \tag{SM1.3}$$

Here, $\mathcal{K}_{\mathrm{long}}$ captures long-range interactions, while $\mathcal{K}_{\mathrm{short}}$ captures short-range effects. The activation function σ, such as `GELU` [46], acts pointwise at each spatial location and on each channel. The residual connection is inspired by residual networks [45] and the ODE interpretation of deep neural networks [30], and it helps stabilize training, especially for deeper architectures.

For the global component of the linear operator, we use a Fourier neural operator (FNO) representation [71, 64, 89, 90, 62]. The corresponding global kernel is represented by a truncated Fourier expansion:

$$(\mathcal{K}_{\mathrm{long}} g)(x) = \sum_{\|k\|_\infty \le k_{\max}} \int_D \kappa_k(x, y) W_k^v \, g(y) \, dy = \sum_{\|k\|_\infty \le k_{\max}} \phi_k(x) \, W_k^v \int_D \overline{\phi_k(y)} \, g(y) \, dy, \tag{SM1.4}$$

where $k \in \mathbb{Z}^d$ denotes a Fourier mode, $\kappa_k(x, y) = e^{2\pi i k \cdot \frac{x-y}{l}}$, $\phi_k(x) = e^{2\pi i k \cdot x / l}$, l is a characteristic length scale of the domain, and $W_k^v \in \mathbb{C}^{d_g \times d_g}$ is the learned complex

matrix associated with mode k. Only modes satisfying $\|k\|_\infty \le k_{\max}$ are retained, while all remaining modes are set to zero. This truncation reduces the number of learnable parameters and yields a low-rank spectral representation of the global operator. When the computational domain D is a fixed periodic box discretized on a structured grid, (SM1.4) can be evaluated efficiently using the fast Fourier transform (FFT):

$$\mathcal{K}_{\text{long}} g = \mathcal{F}^{-1}\Big(R \cdot \big(\mathcal{F} g\big)\Big), \tag{SM1.5}$$

where $\mathcal{F}$ and $\mathcal{F}^{-1}$ denote the channelwise Fourier transform and inverse Fourier transform, respectively. The multiplier R is defined by

$$R(k) = W_k^v \quad \text{for } \|k\|_\infty \le k_{\max}, \qquad R(k) = 0 \quad \text{otherwise.}$$

Thus, the FNO layer applies a learned linear map to each retained Fourier mode and then transforms the result back to physical space. Alternative kernel-based neural operators use kernels that are localized in physical space [2], for example satisfying $\kappa_k(x, y) = 0$ whenever $\|x - y\|_2 > r$ for a prescribed interaction radius $r > 0$, or kernels that are made input-dependent through attention mechanisms [17, 15]. The Fourier basis $\{\phi_k\}$ may also be replaced by multiwavelet bases [38] or Laplace-Beltrami eigenbases [9, 19].

For the local component of the linear operator, we use a pointwise affine map together with a gradient correction [114, 77]:

$$(\mathcal{K}_{\text{short}} g)(x) = W g(x) + b + W^g \text{vec}\big(\nabla_D g(x)\big), \tag{SM1.6}$$

where $W \in \mathbb{R}^{d_g \times d_g}$, $b \in \mathbb{R}^{d_g}$, and $W^g \in \mathbb{R}^{d_g \times d_g d}$ are learned parameters. The pointwise affine term is important because integral operators with continuous kernels are compact on many infinite-dimensional function spaces, whereas the identity operator is not [65, Section 2]. Consequently, a purely integral architecture may not represent identity-type components efficiently. The gradient term is motivated by the structure of PDEs, where differential operators and their compositions appear naturally. Including $\nabla_D g$ therefore helps the model capture local high-frequency features and sharp spatial variations. Near discontinuities, discrete gradients may become large and lead to numerical instability. To regularize this effect, we apply the `SoftSign` activation componentwise, followed by a learned scaling matrix $W^{g'} \in \mathbb{R}^{d_g \times d_g}$:

$$\mathcal{K}_{\text{short}} g(x) = W g(x) + b + W^{g'} \texttt{SoftSign}\Big(W^g \text{vec}\big(\nabla_D g(x)\big)\Big) \quad \text{where } \texttt{SoftSign}(z) = \frac{z}{1 + |z|}. \tag{SM1.7}$$

The smoothed gradient correction in (SM1.7) preserves gradient information while also serving as an indicator of sharp or discontinuous local features in g.

Combining (SM1.4), or equivalently (SM1.5), with (SM1.7) defines the linear component of the neural operator layer. Both components depend implicitly on the computational domain D. The learnable parameters in each layer are collected in $\{W_k^v, W, b, W^g, W^{g'}\}$. The resulting operator is designed at the continuous level and subsequently discretized according to the underlying geometry. We discuss the implementation details in the next section.

SM1.2. Implementation details. The neural operator requires the evaluation of both a global integral operator and a local gradient operator. The corresponding implementation depends on the geometry of the computational domain D.

Structured grids. When the computational domain D is a box discretized on a structured grid, the global integral operator is evaluated using the fast Fourier transform, as in (SM1.5), while the local gradient is approximated using centered finite differences. Specifically, for each spatial direction $i = 1, \dots, d$,

$$\frac{\partial g}{\partial x_i}(x) = \frac{g(x + h_i e_i) - g(x - h_i e_i)}{2h_i}, \tag{SM1.8}$$

where h_i is the mesh spacing in the i-th direction and e_i is the i-th Euclidean basis vector. At grid points adjacent to the boundary, the centered difference is evaluated using periodic wraparound indexing.

Unstructured grids and variable geometries. When the computational domain is general or variable, we discretize it using a point cloud $\{x^{(i)}\}_{i=1}^{n_e} \subset D$, together with mesh connectivity information and quadrature weights $\{\mathrm{d}\Omega_i\}_{i=1}^{n_e}$. In this setting, the global integral operator is evaluated by applying a separable quadrature rule to (SM1.4):

$$(\mathcal{K}_{\mathrm{long}} g)(x) \approx \sum_{\|k\|_\infty \le k_{\max}} \phi_k(x)\, W_k^v \sum_{i=1}^{n_e} \overline{\phi_k(x^{(i)})}\, g(x^{(i)})\, \mathrm{d}\Omega_i. \tag{SM1.9}$$

Thus, for each retained mode k, one first computes the weighted modal coefficient, then applies the learned matrix W_k^v, and finally reconstructs the output at the target point x. For the local gradient, we follow the point-cloud least-squares construction [114]. For each point x, let $\{x^{(j)}\}_{j=1}^{\nu(x)}$ denote its neighboring points, where $\nu(x)$ is the number of neighbors. The discrete gradient $\nabla_D g(x) \in \mathbb{R}^{d_g \times d}$ is defined as the solution of the local least-squares problem

$$\nabla_D g(x) \approx \arg\min_{G \in \mathbb{R}^{d_g \times d}} \sum_{j=1}^{\nu(x)} \left| g(x^{(j)}) - g(x) - G \cdot (x^{(j)} - x) \right|^2. \tag{SM1.10}$$

We introduce the geometry matrix and the feature-difference matrix

$$A(x) = \begin{bmatrix} x^{(1)} - x & \cdots & x^{(\nu(x))} - x \end{bmatrix} \in \mathbb{R}^{d \times \nu(x)}, \tag{SM1.11}$$

$$B_g(x) = \begin{bmatrix} g(x^{(1)}) - g(x) & \cdots & g(x^{(\nu(x))}) - g(x) \end{bmatrix} \in \mathbb{R}^{d_g \times \nu(x)}. \tag{SM1.12}$$

Then (SM1.10) can be written equivalently as

$$\nabla_D g(x) \approx \arg\min_{G \in \mathbb{R}^{d_g \times d}} \|GA(x) - B_g(x)\|_F^2 \tag{SM1.13}$$

and the least-squares solution is

$$\nabla_D g(x) = B_g(x)\, A(x)^\dagger, \tag{SM1.14}$$

where $A(x)^\dagger \in \mathbb{R}^{\nu(x) \times d}$ is the pseudoinverse of $A(x)$, precomputed from the local geometry. If the j-th row of $A(x)^\dagger$ is denoted by $A_j(x)^\dagger \in \mathbb{R}^d$, then (SM1.14) can be evaluated in a single message-passing step:

$$\nabla_D g(x) = \sum_{j=1}^{\nu(x)} \big(g(x^{(j)}) - g(x)\big) A_j(x)^\dagger. \tag{SM1.15}$$

When D is a surface embedded in $\mathbb{R}^d$, as in the vehicle aerodynamics case, this procedure computes a least-squares gradient in ambient coordinates, which gives a tangential gradient approximation on the surface.

Remark SM1.1 (Boundary conditions). In the architectures considered here, boundary conditions are generally not imposed strongly by the neural operator layers themselves, but are instead learned from the training data. Periodic boundary conditions are an exception, since they are naturally compatible with the Fourier representation in (SM1.5) and with periodic wraparound indexing. For nonperiodic box domains, we follow the padding strategy used in the FNO [71]: the domain is enlarged by 10% in each coordinate direction and the data are zero-padded to provide a simple periodic extension in the spirit of Fourier continuation [14]. For unstructured grids and variable geometries, the characteristic length l in the Fourier features is chosen to be approximately twice the size of the bounding box, following [41]. Boundary conditions may also be imposed strongly, for example by using bases adapted to prescribed boundary conditions [49, 6, 76, 19, 103]. In the present work, we do not consider problems with variable boundary conditions. When such conditions vary across samples, they can be incorporated into the input data a, for example through additional boundary channels; see [117].

SM1.3. Cost-accuracy behavior. Finally, we discuss the cost-accuracy behavior of the neural operator described above. We focus on online inference cost. Floating-point operation counts are estimated under the standard model in which each multiplication or addition is counted as one flop, while memory traffic, communication, and parallelization overhead are ignored.

Let n_e denote the total number of grid points or point-cloud degrees of freedom, and let

$$K := (2k_{\max} + 1)^d$$

denote the number of retained modes in the truncated spectral representation. Recall that d_g and L denote the hidden channel dimension and the number of neural operator layers, respectively. Together, n_e, K, d_g, and L determine the leading-order inference cost. The precise cost depends on the geometry of the computational domain. On a structured periodic grid, the global operator is evaluated by FFT, whereas on an unstructured point cloud it is evaluated by the separable quadrature formula (SM1.9). The local gradient correction is computed by centered finite differences on structured grids and by least-squares message passing on unstructured grids. Since the local pseudoinverses $A(x)^\dagger$ are precomputed, the two gradient implementations have comparable leading-order complexity.

Lifting and projection. Assume that the lifting map $\mathcal{P}$ is a pointwise affine map. Then its cost is

$$C_{\text{lift}} = 2n_e d_a d_g.$$

Assume further that the projection map $\mathcal{Q}$ is a two-layer pointwise multilayer perceptron $\mathbb{R}^{d_g} \to \mathbb{R}^{d_g} \to \mathbb{R}^{d_u}$ with one pointwise activation in the hidden layer. Then its cost is

$$C_{\text{proj}} = 2n_e d_g^2 + 2n_e d_g d_u + c_\sigma n_e d_g.$$

Structured-grid cost. For a structured grid, the global integral operator is evaluated using the FFT. If one complex FFT of size n_e costs $5n_e \log_2 n_e$ flops, then one forward and one inverse transform on all d_g channels cost $10 d_g n_e \log_2 n_e$. For the retained modes, each complex multiplication is counted as 6 flops, hence each complex matrix-vector product costs $d_g(6d_g + 2(d_g - 1)) = 8d_g^2 - 2d_g$ flops. Hence the total cost of the structured global operator is

$$10 d_g n_e \log_2 n_e + K(8d_g^2 - 2d_g). \tag{SM1.16}$$

For the local operator (SM1.6), the pointwise affine term costs $2d_g^2 n_e$. The centered finite difference gradient (SM1.8) requires one subtraction and one multiplication per point, per channel, and per spatial direction, and therefore costs $2dd_g n_e$. Applying the gradient map $W^g \in \mathbb{R}^{d_g \times (d_g d)}$ costs $(2d\, d_g^2 - d_g)n_e$. Applying the componentwise SoftSign nonlinearity costs $3d_g n_e$. Applying the scaling matrix $W^{g'} \in \mathbb{R}^{d_g \times d_g}$ costs $(2\, d_g^2 - d_g)n_e$. Adding the pointwise branch and the gradient-correction branch costs $d_g n_e$, and combining the local contribution with the global contribution costs another $d_g n_e$. Finally, evaluating the outer activation and applying the residual connection cost $c_\sigma d_g n_e + d_g n_e$.

Therefore, one structured-grid neural operator layer costs

(SM1.17)
$$C_{\text{layer}} = 10 d_g n_e \log_2 n_e + K(8d_g^2 - 2d_g) + ((2d+4)d_g^2 + (2d + 4 + c_\sigma)d_g)n_e.$$

For an L-layer structured-grid neural operator, the inference cost satisfies

$$C_{\text{NO}}^{\text{infer}} = C_{\text{lift}} + C_{\text{proj}} + LC_{\text{layer}}. \tag{SM1.18}$$

When K, n_e, and d_g are the dominant parameters, this reduces to

$$C_{\text{NO}}^{\text{infer}} = \mathcal{O}\big(10Ld_g n_e \log_2 n_e + 8LKd_g^2 + (2 + (2d+4)L)d_g^2 n_e\big). \tag{SM1.19}$$

Unstructured-grid cost. For a general point cloud, the global integral operator is evaluated using (SM1.9). To reduce memory usage, especially in the case of variable computational domains, we evaluate the basis functions $\phi_k(x)$ online rather than storing them for all modes and all points. For each retained mode k, evaluating the basis on all points costs $4dn_e$ flops, forming the weighted modal coefficient costs $4d_g n_e$ flops, applying the learned matrix W_k^v costs $8d_g^2 - 2d_g$ flops, and reconstructing the output at all points costs $8d_g n_e$ flops. Hence the total cost of the unstructured global operator is

$$K(12d_g + 4d)n_e + K(8d_g^2 - 2d_g). \tag{SM1.20}$$

For the local operator, the only difference from the structured-grid case lies in the gradient evaluation. In the message-passing gradient formula (SM1.15), each directed neighbor contribution requires d_g flops to form the feature difference and $2d\, d_g$ flops to apply the local aggregation. Thus, the gradient cost is

$$2(2d+1)d_g\, n_f. \tag{SM1.21}$$

The remaining pointwise operations are the same as in the structured-grid case. The pointwise affine term costs $2d_g^2 n_e$; the gradient map W^g costs $(2d\, d_g^2 - d_g)n_e$; the componentwise SoftSign costs $3d_g n_e$; the scaling matrix $W^{g'}$ costs $(2d_g^2 - d_g)n_e$; the branch additions cost $2d_g n_e$; and the outer activation and residual connection cost $c_\sigma d_g n_e + d_g n_e$.

Therefore, one unstructured-grid neural operator layer costs [1]

(SM1.23)
$$C_{\text{layer}} = K(12d_g + 4d)n_e + K(8d_g^2 - 2d_g) + 2(2d+1)d_g\, n_f + \big((2d+4)d_g^2 + (4 + c_\sigma)d_g\big)n_e.$$

[1] When the surface normals are incorporated in each layer following the geometric encoding strategy of [41], the global operator costs

$$K(12d_g + 4d)n_e + K(8d_g^2 - 2d_g) + 2d_g(d + 2d_g(d+1) - 1)n_e.$$

In practice, the neighborhood size is typically uniformly bounded and is comparable to that of a structured grid, so that $n_f \approx dn_e$. Hence, for an L layer unstructured-grid neural operator, the total inference cost up to low-order terms is

$$\begin{aligned} C_{\mathrm{NO}}^{\mathrm{infer}} &= C_{\mathrm{lift}} + C_{\mathrm{proj}} + LC_{\mathrm{layer}} \\ &= \mathcal{O}\big(12KLd_g n_e + 8LKd_g^2 + (2+(2d+4)L)d_g^2 n_e\big). \end{aligned} \tag{SM1.24}$$

For fixed K, d_g, and L, the structured-grid cost (SM1.19) is quasilinear in n_e, whereas the unstructured-grid cost (SM1.24) is linear in n_e. However, the associated prefactors may nevertheless be substantial because they depend strongly on the hidden channel dimension, the number of retained modes, and the network depth.

Predictive accuracy. The predictive accuracy of neural operators remains less well understood than that of classical numerical solvers. A useful heuristic model [78, 50] for the in-distribution test error is

$$\mathbb{E}_{(a,D)\sim\mu}\Big(\frac{\|\mathcal{G}^\dagger(a,D)-\mathcal{G}(a,D;\theta_N)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a,D)\|_{\mathcal{U}}}\Big) \approx c_{\mathrm{disc}} h_{\mathrm{eff}}^{q} + c_{\mathrm{data}} N^{-\beta} + \varepsilon_{\mathrm{opt}}, \tag{SM1.25}$$

where θ_N denotes parameters trained on N samples. The terms in this decomposition correspond to distinct sources of error and have been analyzed only in simplified settings. The first term represents discretization or representation error. Here h_{eff} denotes an effective resolution scale. When $k_{\max}$ is sufficiently large, the Fourier representation can resolve the underlying spatial discretization, and h_{eff} may be identified with the physical mesh size [63, Theorem 24][59]. When $k_{\max}$ is more restrictive, the dominant error may instead arise from Fourier truncation, in which case $h_{\mathrm{eff}} \approx k_{\max}^{-1}$ may be regarded as the effective resolution scale [41, Theorem 2.1]. Related representation errors also arise from encoder-decoder architectures, such as PCA-based neural operators [6, Theorem 3.4]. The exponent $q > 0$ should therefore be interpreted as an effective approximation exponent, reflecting both the regularity of the target operator and the approximation properties of the neural operator architecture. The second term represents the statistical error arising from the approximation of the population risk

$$\mathcal{J}_\infty(\theta) := \mathbb{E}_{(a,D)\sim\mu}\Big(\frac{\|\mathcal{G}^\dagger(a,D)-\mathcal{G}(a,D;\theta)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a,D)\|_{\mathcal{U}}}\Big) \tag{SM1.26}$$

by the empirical risk (SM1.27)

$$\mathcal{J}_N(\theta) := \mathbb{E}_{(a,D)\sim\mu^N}\Big(\frac{\|\mathcal{G}^\dagger(a,D)-\mathcal{G}(a,D;\theta)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a,D)\|_{\mathcal{U}}}\Big) = \frac{1}{N}\sum_{i=1}^{N}\frac{\|\mathcal{G}^\dagger(a_i,D_i)-\mathcal{G}(a_i,D_i;\theta)\|_{\mathcal{U}}}{\|\mathcal{G}^\dagger(a_i,D_i)\|_{\mathcal{U}}} \tag{SM1.27}$$

The local operator and the remaining terms cost

$$2(2d+1)d_g\, n_f + \big((2d+4)d_g^2 + (4+c_\sigma)d_g\big)n_e + d_g(4d_g + 2(d^2+d)+2)n_e$$

The cost for computing the gradient of the surface normal is $2(2d+1)dn_f$. Hence, for an L layer unstructured-grid neural operator, the total inference cost up to low-order terms is

$$\begin{aligned} C_{\mathrm{NO}}^{\mathrm{infer}} &= C_{\mathrm{lift}} + C_{\mathrm{proj}} + LC_{\mathrm{layer}} \\ &= \mathcal{O}\big(12KLd_g n_e + 8LKd_g^2 + (2+(6d+12)L)d_g^2 n_e\big). \end{aligned} \tag{SM1.22}$$

based on N training samples with $\mu^N = \frac{1}{N}\sum_{j=1}^{N}\delta_{(a_j,D_j)}$. The exponent $\beta > 0$ generally depends on the regularity of the target solution operator and on the input distribution; see, for example, [28, Theorem 1.3][98, Corollary 2.8][54, Theorem 4.9][75][74, Theorem 4]. The final term, ε_{opt}, accounts for residual error because training only approximately minimizes the nonconvex empirical objective. This residual may arise from finite optimization time, stochastic gradient noise, or convergence to a suboptimal stationary point [10, 23].

Accordingly, in the present work, we focus primarily on empirical predictive accuracy rather than on a formal a priori error analysis.

SM2. Classical numerical solvers. In this section, we derive detailed floating-point operation counts for the classical numerical solvers used in the three benchmarks; comparable derivations are not readily available in the literature. Subsection SM2.1 considers finite elements with geometric multigrid for two-dimensional Darcy flow, subsection SM2.2 considers a Fourier pseudospectral method with fourth-order Runge-Kutta time integration for two-dimensional incompressible Navier-Stokes flow, and subsection SM2.3 considers a finite-volume RANS solver for vehicle aerodynamics. Throughout, each multiplication or addition is counted as one floating-point operation, while memory traffic, communication, and parallelization overhead are excluded.

SM2.1. Darcy flow. Assume that the multigrid hierarchy consists of $L+1$ levels, indexed by $l = 0, 1, \ldots, L$, where level 0 is the finest grid and level L is the coarsest grid. We assume that each coarsening step reduces the number of elements in each spatial direction by a factor of 2, and that the linear system on the coarsest grid is solved directly.

At level l, the domain is partitioned into an $n_l \times n_l$ quadrilateral mesh. For conforming Q_1 finite elements with homogeneous Dirichlet boundary conditions, the number of active degrees of freedom is

$$n_{e,l} = (n_l - 1)^2.$$

Since each interior degree of freedom couples to the degrees of freedom in the surrounding 3×3 nodal stencil, the number of nonzero entries in the stiffness matrix A_l is

$$\operatorname{nnz}(A_l) = (3n_l - 5)^2.$$

Let P_l denote the prolongation matrix from level $l+1$ to level l, that is, from the coarse grid to the next finer grid. For bilinear interpolation on a uniform quadrilateral grid,

$$\operatorname{nnz}(P_l) = 9n_{e,l+1}.$$

The restriction matrix from level l to level $l+1$ is taken to be P_l^T.

The stiffness matrices on all levels, the prolongation operators between adjacent levels, and the finest-grid load vector are assembled in the assembly phase. The assembly cost therefore satisfies

$$C_{\text{assemble}} \approx \sum_{l=0}^{L} 156n_l^2 + 60n_0^2 + \sum_{l=1}^{L} 9n_{e,l} \approx 271n_e. \tag{SM2.1}$$

Here 156 is the per-element cost for assembling the stiffness matrix, 60 is the per-element cost for assembling the finest-grid load vector, and the third term bounds the cost of constructing the prolongation operators. The constants 156 and 60 are

obtained from a direct quadrature-level flop count for Q_1 quadrilateral elements with a 2×2 tensor-product Gaussian rule. At each quadrature point, evaluating the nodal Q_1 coefficient a_h requires 4 multiplications and 3 additions, for a total of 7 flops. Updating the 4×4 local stiffness matrix requires one multiplication and one addition per entry, namely 32 flops. Thus the stiffness assembly costs $7 + 32 = 39$ flops per quadrature point, and hence $4 \times 39 = 156$ flops per element. Similarly, evaluating the nodal Q_1 source term f_h costs 7 flops per quadrature point, while updating the four entries of the local load vector costs 8 flops. Thus the load-vector assembly costs $7 + 8 = 15$ flops per quadrature point, and hence $4 \times 15 = 60$ flops per element. Only the finest-grid load vector is assembled from the source term; on coarser levels, the right-hand sides are obtained by restricting residuals during the multigrid iteration.

We next estimate the floating-point cost of one multigrid V-cycle. At each non-coarsest level $l = 0, 1, \ldots, L-1$, the algorithm consists of ν_1 pre-smoothing steps, one residual computation, one restriction to the next coarser level, one prolongation of the coarse-grid correction, and ν_2 post-smoothing steps. Let $\nu := \nu_1 + \nu_2$ denote the total number of smoothing steps per level. For Richardson-Jacobi smoothing,

$$u_{h,l} \leftarrow u_{h,l} + \omega D_l^{-1}(b_l - A_l u_{h,l}),$$

where b_l is the current right-hand side (equal to the restricted residual), D_l is the diagonal of A_l, and ω is the relaxation parameter. One smoothing step costs $2\ \mathrm{nnz}(A_l) + 4n_{e,l}$, where $2\ \mathrm{nnz}(A_l)$ is the cost of the sparse matrix-vector product, and $4n_{e,l}$ accounts for residual formation, diagonal scaling, and the vector update. The residual is then computed as

$$r_l = b_l - A_l u_{h,l},$$

which costs $2\ \mathrm{nnz}(A_l) + n_{e,l}$. The restriction step

$$b_{l+1} = P_l^T r_l$$

projects the residual onto the right-hand side at the next coarser level and costs $2\ \mathrm{nnz}(P_l)$ flops. The prolongation of the coarse-grid correction e_{l+1} back to level l, followed by the correction step

$$u_{h,l} \leftarrow u_{h,l} + P_l e_{l+1}$$

costs $2\ \mathrm{nnz}(P_l) + n_{e,l}$. On the coarsest level, the linear system is solved directly by LU factorization. The corresponding cost is approximately $n_{e,L}^3$, which is $\mathcal{O}(1)$ when the coarsest grid is fixed. Therefore, the cost of one V-cycle is

(SM2.2)
$$\begin{aligned} C_{\mathrm{solve}} &\approx \sum_{l=0}^{L-1} \Big(\nu\big(2\ \mathrm{nnz}(A_l) + 4n_{e,l}\big) + \big(2\ \mathrm{nnz}(A_l) + n_{e,l}\big) + \big(4\ \mathrm{nnz}(P_l) + n_{e,l}\big) \Big) + n_{e,L}^3 \\ &\approx \big(\frac{88}{3}\nu + \frac{116}{3}\big) n_e. \end{aligned}$$

Combining the assembly cost with the solve cost of m multigrid V-cycles yields
(SM2.3)
$$C_{\mathrm{solver}} = C_{\mathrm{assemble}} + mC_{\mathrm{solve}} \approx 271 n_e + m\big(\frac{88}{3}\nu + \frac{116}{3}\big) n_e = \left(c_{\mathrm{assemble}} + c_{\mathrm{solve}}\, m\right) n_e.$$

Therefore, $c_{\mathrm{assemble}} = 271$ and $c_{\mathrm{solve}} = \frac{88}{3}\nu + \frac{116}{3}$. For the common choice $\nu_1 = \nu_2 = 3$, both coefficients are on the order of a few hundred.

SM2.2. Incompressible Navier-Stokes flow. Assume that a two-dimensional complex FFT on an $n_e = n \times n$ grid costs $5n_e \log_2 n_e$. Given the Fourier coefficients $\widehat{\omega}$, the right-hand side is evaluated in spectral form as

$$\widehat{\mathcal{N}}(\widehat{\omega})_k = -\widehat{(v \cdot \nabla\omega)}_k - \nu|2\pi k|^2 \widehat{\omega}_k + \widehat{f}_k,$$

where $k \in \mathbb{Z}^2$ is the integer Fourier index and $2\pi k$ is the corresponding physical wavenumber on $D = [0,1]^2$. The nonlinear term $v \cdot \nabla\omega$ is evaluated pseudo-spectrally: v and $\nabla\omega$ are transformed to physical space, multiplied pointwise, and then transformed back to Fourier space. For $|k| \neq 0$, the velocity and vorticity derivatives are computed in Fourier space as

$$\widehat{v}_1 = \frac{i2\pi k_2}{|2\pi k|^2}\widehat{\omega}, \qquad \widehat{v}_2 = -\frac{i2\pi k_1}{|2\pi k|^2}\widehat{\omega}, \qquad \widehat{\partial_x\omega} = i2\pi k_1\widehat{\omega}, \qquad \widehat{\partial_y\omega} = i2\pi k_2\widehat{\omega}.$$

These four Fourier multiplier operations cost approximately $24n_e$ flops if each complex multiplication is counted as 6 flops. We then apply four inverse FFTs to obtain $v_1, v_2, \partial_x\omega, \partial_y\omega$ in physical space. The nonlinear term

$$v \cdot \nabla\omega = v_1\partial_x\omega + v_2\partial_y\omega$$

is formed pointwise at a cost of $3n_e$ flops. A forward FFT is then used to transform the nonlinear term back to Fourier space. Finally, the viscous term $-\nu|2\pi k|^2\widehat{\omega}$, the forcing term, and the nonlinear term are combined in Fourier space, at a cost of approximately $10n_e$ flops. Dealiasing using the 2/3 rule [92] is applied at each right-hand-side evaluation; for a simple spectral mask, this costs approximately $2n_e$ flops. Therefore, one right-hand-side evaluation costs

$$C_{\mathrm{rhs}} \approx 25n_e \log_2 n_e + (24 + 3 + 10 + 2)n_e = 25n_e \log_2 n_e + 39n_e. \tag{SM2.4}$$

One RK4 time step requires four right-hand-side evaluations,

$$\widehat{k}_1 = \widehat{\mathcal{N}}(\widehat{\omega}^n), \quad \widehat{k}_2 = \widehat{\mathcal{N}}(\widehat{\omega}^n + \tfrac{\Delta t}{2}\widehat{k}_1), \quad \widehat{k}_3 = \widehat{\mathcal{N}}(\widehat{\omega}^n + \tfrac{\Delta t}{2}\widehat{k}_2), \quad \widehat{k}_4 = \widehat{\mathcal{N}}(\widehat{\omega}^n + \Delta t\,\widehat{k}_3),$$

followed by the spectral-space update

$$\widehat{\omega}^{n+1} = \widehat{\omega}^n + \frac{\Delta t}{6}(\widehat{k}_1 + 2\widehat{k}_2 + 2\widehat{k}_3 + \widehat{k}_4).$$

The RK4 stage-state formation and final update require approximately $28n_e$ additional flops. If n_t time steps are used to reach the final time T, the total floating-point cost is

$$\begin{aligned} C_{\mathrm{solver}} &\approx n_t(4C_{\mathrm{rhs}} + 28n_e) = n_t\,(100n_e \log_2 n_e + 184n_e) \\ &= n_t\,(c_{\mathrm{assemble}}\, n_e \log_2 n_e + c'_{\mathrm{assemble}}\, n_e)\,. \end{aligned} \tag{SM2.5}$$

Therefore, $c_{\mathrm{assemble}} = 100$ and $c'_{\mathrm{assemble}} = 184$, both constants are on the order of a few hundred.

The time step is chosen to satisfy both the advective CFL condition and the explicit viscous stability condition

$$\Delta t \leq \min\left\{\alpha_{\mathrm{CFL}}\frac{h}{\|v\|_{L^\infty}}, \frac{\alpha_{\mathrm{RK4}}}{\nu(2\pi k_{\max})^2}\right\},$$

here α_{CFL} is an $O(1)$ CFL constant, $\|v\|_{L^\infty}$ is the maximum velocity magnitude, $\alpha_{\mathrm{RK4}} \approx 2.785$ is the stability limit of classical RK4 [40], and $k_{\max}$ is the largest retained Fourier index. In our setting, the viscosity is small, so the advective CFL condition dominates. For fixed final time $T = 1$ and $\|v\|_{L^\infty} = O(1)$, this gives $\Delta t = O(h)$ and $n_t = O(n)$. Thus,

$$C_{\mathrm{solver}} \approx c_{\mathrm{assemble}}\, n_e^{3/2} \log_2 n_e + c'_{\mathrm{assemble}}\, n_e^{3/2}. \tag{SM2.6}$$

SM2.3. Vehicle aerodynamics. The nonlinear steady RANS-k-ω SST system consists of the three momentum equations, the incompressibility constraint, and the two turbulence transport equations for k and ω. In `OpenFOAM`, it is solved using a segregated SIMPLE fixed-point iteration. At each SIMPLE iteration, `OpenFOAM` assembles six separate linearized systems for the momentum, pressure-correction, k, and ω equations, which are then solved sequentially in a block-segregated manner. The momentum predictor is first solved using the current pressure, turbulent viscosity, and face fluxes; the pressure-correction equation is then solved to enforce discrete mass conservation and update the velocity and flux fields. The corrected velocity and flux are subsequently used to assemble and solve the k- and ω-transport equations, after which the eddy viscosity is updated for the next SIMPLE iteration.

In the `OpenFOAM` implementation, the implicit finite volume matrices are sparse and retain only the nearest-neighbor stencil induced by the mesh faces. For the convective terms, only the lower-order part of the discretization is treated implicitly, while the gradient-based linear-upwind correction is handled explicitly as a source contribution. Similarly, for corrected diffusion schemes, the implicit matrix contains only the uncorrected Gauss-Laplacian contribution (see (SM2.9)), while the non-orthogonal correction terms are treated explicitly in the source term. Let n_c denote the number of control volumes and n_f the number of internal faces. Then each scalar finite volume matrix A has approximately

$$\mathrm{nnz}(A) = n_c + 2n_f,$$

since each cell contributes one diagonal entry and each internal face contributes two off-diagonal couplings. These matrices are reassembled at each SIMPLE iteration.

We estimate the cost of one SIMPLE iteration by separating it into an assembly cost and a linear-solve cost. Consider first one representative scalar convection-diffusion equation. The implicit matrix contains only the low-order convective contribution and the uncorrected diffusive contribution. Since both are assembled by looping over the internal faces, we estimate their costs as approximately $4n_f$ and $3n_f$ flops, respectively. The remaining assembly cost comes from explicit correction and source terms. A Gauss linear gradient reconstruction for a cell-centered scalar ϕ_e is given by

$$\nabla\phi_e \approx \frac{1}{|\Omega_e|} \sum_{f \subset \partial\Omega_e} \phi_f s_f,$$

where $|\Omega_e|$ is the cell volume, ϕ_f is the face-interpolated value, and s_f is the face area vector. We estimate this cost as approximately $12n_f + 3n_c$. For the cell-limited Gauss gradient, the unlimited gradient is rescaled as

$$\widetilde{\nabla\phi_e} = \lambda_e \nabla\phi_e \qquad \phi_f \approx \phi_e + \widetilde{\nabla\phi_e} \cdot (x_f - x_e), \tag{SM2.7}$$

where $0 \le \lambda_e \le 1$ is chosen to prevent the reconstructed face values ϕ_f from creating new extrema. We estimate the additional cost of limiter construction and gradient

rescaling as $18n_f + 5n_c$. Using these gradients, the explicit linear-upwind correction at face f is

$$\delta\phi_f^{\mathrm{up}} = \widetilde{\nabla\phi}_{e_{\mathrm{up}}} \cdot (x_f - x_{e_{\mathrm{up}}}), \tag{SM2.8}$$

and contributes $F_f\,\delta\phi_f^{\mathrm{up}}$ to the source term, where F_f is the face flux and e_{up} is the upwind cell. We estimate this cost as $11n_f + n_c$. For corrected diffusion, the diffusive face flux is decomposed into an implicit orthogonal part and an explicit non-orthogonal correction:

$$\begin{aligned} \nabla \cdot (\nu \nabla \phi_e) &\approx \frac{1}{|\Omega_e|} \sum_{f \subset \partial\Omega_e} \nu_f (\nabla\phi)_f \cdot s_f \\ &\approx \frac{1}{|\Omega_e|} \sum_{f \subset \partial\Omega_e} \Big(\nu_f \alpha_f (\phi_{e'} - \phi_e) + \nu_f (s_f - \alpha_f d_{e,e'}) (\nabla\phi)_f \Big), \end{aligned} \tag{SM2.9}$$

where $d_{e,e'} = x_{e'} - x_e$ is the vector connecting the neighboring cell centers, and α_f is the scalar coefficient of the orthogonal two-point diffusive flux. The second term is treated explicitly, and we estimate its cost as $16n_f + n_c$. Therefore, for one representative scalar equation, the assembly cost is approximately

$$7n_f + (12n_f + 3n_c) + (18n_f + 5n_c) + (11n_f + n_c) + (16n_f + n_c) = 64n_f + 10n_c. \tag{SM2.10}$$

Counting the three velocity components, the pressure-correction equation, and the two turbulence equations as six scalar-equivalent systems gives the rough assembly estimate

$$C_{\mathrm{assemble}} \approx 6(64n_f + 10n_c) = 384n_f + 60n_c. \tag{SM2.11}$$

For the solve cost, each scalar finite volume matrix has approximately $n_c + 2n_f$ nonzero entries. One symmetric Gauss-Seidel sweep therefore costs approximately $4(n_c + 2n_f)$ flops. If each scalar system requires m inner iterations to reach the prescribed residual tolerance, and if all six systems are modeled with the same per-iteration cost, then

$$C_{\mathrm{solve}} \approx 6m\,4(n_c + 2n_f) = 24m\,n_c + 48m\,n_f.$$

In practice, the pressure-correction equation is solved with the geometric agglomerated algebraic multigrid solver `GAMG`, while the velocity and turbulence equations are solved using `smoothSolver` with symmetric Gauss-Seidel smoothing. Their actual constants therefore differ, and the simplified model above should be understood only as a leading-order cost estimate.

Finally, if n_t SIMPLE iterations are required, and we use the approximation $n_f \approx 3n_c$ for shape-regular finite volume meshes, then the total cost is approximately

$$C_{\mathrm{solver}} \approx n_t(C_{\mathrm{assemble}} + C_{\mathrm{solve}}) = n_t(1212n_c + 168m\,n_c) = n_t(c_{\mathrm{assemble}} n_c + c_{\mathrm{solve}} m\,n_c),$$

where $c_{\mathrm{assemble}} = 1212$ and $c_{\mathrm{solve}} = 168$, which are on the order of a few thousand and a few hundred, respectively. For the inner linear solves, we follow the settings of the official `OpenFOAM` 12 `drivaerFastback` case. The pressure, velocity, and turbulence equations use an absolute tolerance of 10^{-6} and a relative tolerance of 0.1. Our solver logs show that each linear system typically requires two to three inner iterations. To avoid overstating the classical solver cost, we use $m = 2$ as a conservative effective value in the floating-point cost estimate.

SM3. Additional numerical results. This section provides additional numerical results that complement the comparisons in the main paper. First, we test the sensitivity of the Darcy flow results to the neural operator architecture by repeating the experiment with a standard Fourier neural operator. Second, we examine the sensitivity of the Navier–Stokes results to the choice of time integrator. Finally, we report the configuration and results of a representative `OpenFOAM` calculation for the canonical `drivaerFastback` geometry.

SM3.1. Architecture sensitivity for Darcy flow. To assess sensitivity to architecture, we repeat the Darcy flow experiment using a standard Fourier neural operator (FNO) [71], with the same data, training protocol, and evaluation procedure. As shown in Figure SM1, the qualitative conclusion is unchanged: geometric multigrid requires less floating-point work than either neural operator, whereas both neural operators achieve shorter wall-clock runtimes over the accuracy range they attain, owing to the efficient execution of dense tensor operations on modern hardware. These results suggest that the observed behavior is robust to this architectural modification. Because the underlying acceleration mechanisms are shared by many neural operator architectures, we expect similar qualitative behavior more broadly; establishing this generality, however, requires comparisons across additional architectures and problems.

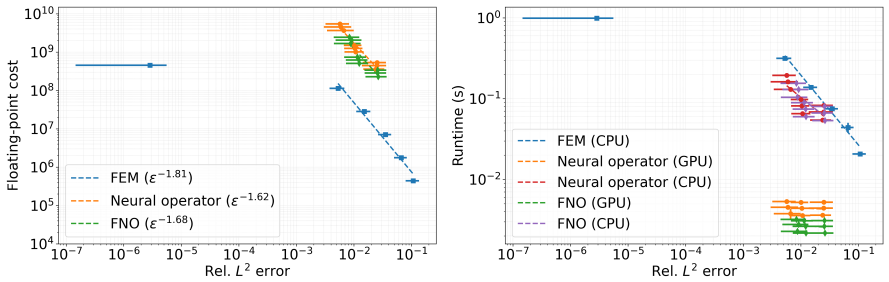

FIG. SM1. *Darcy flow: cost–accuracy comparison among finite elements with geometric multigrid (FEM), the modified Fourier neural operator (Neural operator), and the original Fourier neural operator (FNO). Left: floating-point operation count versus relative L^2 error. Right: wall-clock runtime in seconds versus relative L^2 error for CPU and GPU implementations. Error bars denote one standard deviation over the test samples.*

SM3.2. Time integrator sensitivity for Navier-Stokes flow. To assess sensitivity to the choice of time integrator, we repeat the Navier-Stokes experiment using a first-order exponential time differencing scheme (ETD1) and a fourth-order exponential time differencing Runge-Kutta scheme (ETDRK4) [27], while retaining the same spatial discretization, data, and evaluation protocol. In our experiments, ETD1 is unstable at both the time step used for RK4 and ETDRK4 and at half that value. We therefore use $\Delta t_{\mathrm{ETD1}} = \frac{\Delta t_{\mathrm{RK4}}}{4} = \frac{\Delta t_{\mathrm{ETDRK4}}}{4}$. ETD1 consequently takes four times as many time steps but requires only one nonlinear right-hand-side evaluation per step, whereas RK4 and ETDRK4 require four such evaluations. The leading-order floating-point costs of the three methods are therefore comparable under these settings.

As shown in Figure SM2, ETD1 yields larger trajectory prediction errors at comparable floating-point work, reflecting its first-order temporal accuracy and underscoring the importance of higher-order time integration. ETDRK4 and RK4 exhibit

similar cost-accuracy behavior. In this convection dominated regime, exact treatment of the viscous linear term provides little additional benefit because the time step remains governed primarily by the explicitly treated nonlinear advection. Consequently, the qualitative comparison with the neural operator is unchanged by the choice among the time integrators considered here.

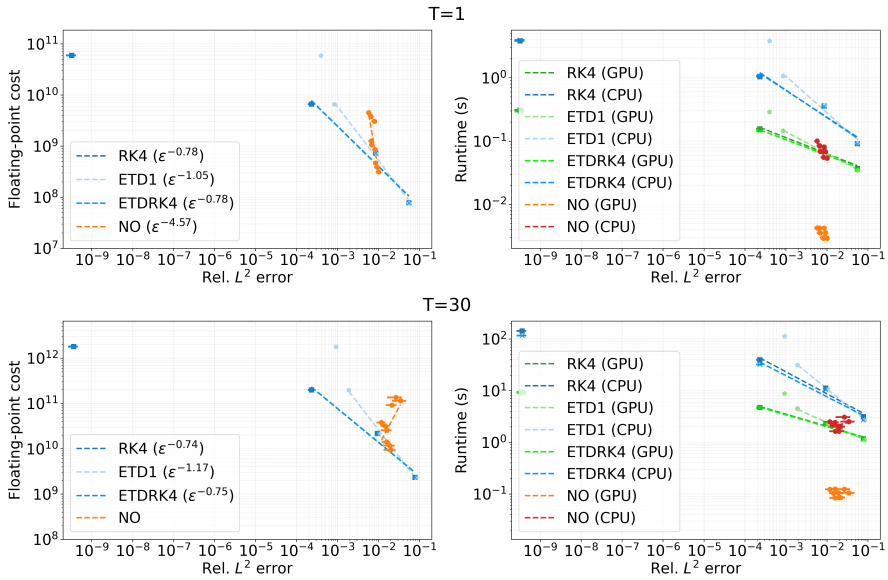

FIG. SM2. *Time integrator sensitivity for incompressible Navier-Stokes flow: cost-accuracy comparisons among Fourier pseudospectral solvers using RK4, ETD1, and ETDRK4 and the neural operator (NO). Top: teacher-forced one-step evaluation ($T = 1$), with each reference state $\omega(t)$, $t = 0, \ldots, 29$,used to predict $\omega(t + 1)$. Bottom: recurrent rollout from $t = 0$ through $T = 30$. Left: floating-point operation count versus relative L^2 error. Right: wall-clock runtime in seconds versus relative L^2 error for CPU and GPU implementations. Error bars denote one standard deviation over the test samples.*

SM3.3. Representative `OpenFOAM` vehicle configuration. The vehicle aerodynamics results reported in the main paper are averaged over the six test geometries. To illustrate the scale of an individual calculation and facilitate reproducibility, Table SM1 reports the mesh sizes, iteration counts, CPU allocations, runtimes, and relative errors for the canonical `drivaerFastback` geometry. These values are close to the corresponding test set averages reported in Table 4.

Although `OpenFOAM` runtimes and numerical errors vary across geometries, this variability can be monitored using mesh quality measures, residual histories, and the stabilization of aerodynamic force coefficients. These diagnostics make the sources of error in the classical solver more readily interpretable than those in the neural operator predictions. Moreover, because the classical solver is not trained on a finite dataset, it is not subject to out-of-distribution errors in the same statistical sense as the neural operator.

REFERENCES

[1] B. ALKIN, M. BLEEKER, R. KURLE, T. KRONLACHNER, R. SONNLEITNER, M. DORFER, AND J. BRANDSTETTER, *AB-UPT: Scaling neural CFD surrogates for high- fidelity automo-*

Configuration	Large reference	Large	Medium	Small
Control volumes n_c	22,551,532	22,551,532	2,982,335	440,405
SIMPLE iterations n_t	7000	2000	1000	1000
CPU cores	256	256	32	4
Runtime (s)	6424	1868	975	557
Relative L^1 error	Reference	11.46%	16.31%	23.62%

TABLE SM1

`OpenFOAM` configurations and results for prediction of surface pressure coefficient C_p in the canonical `drivaerFastback` benchmark. Errors are measured relative to the reference solution computed on the large mesh using 7000 SIMPLE iterations.

tive aerodynamics simulations via anchored- branched universal physics transformers, Transactions on Machine Learning Research, (2025), https://openreview.net/forum?id=nwQ8nitlTZ.

[2] A. Anandkumar, K. Azizzadenesheli, K. Bhattacharya, N. Kovachki, Z. Li, B. Liu, and A. Stuart, *Neural operator: Graph kernel network for partial differential equations*, in ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations, 2019, https://openreview.net/forum?id=fg2ZFmXFO3.

[3] D. Aristoff and W. Bangerth, *A benchmark for the bayesian inversion of coefficients in partial differential equations*, SIAM Review, 65 (2023), pp. 1074–1105.

[4] I. Babuška and J. Osborn, *Can a finite element method perform arbitrarily badly?*, Mathematics of computation, 69 (2000), pp. 443–462.

[5] S. Balay, S. Abhyankar, M. Adams, J. Brown, P. Brune, K. Buschelman, L. Dalcin, A. Dener, V. Eijkhout, W. Gropp, et al., *PETSc users manual*, 2019.

[6] K. Bhattacharya, B. Hosseini, N. B. Kovachki, and A. M. Stuart, *Model reduction and neural networks for parametric PDEs*, The SMAI journal of computational mathematics, 7 (2021), pp. 121–157.

[7] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian, *Accurate medium-range global weather forecasting with 3d neural networks*, Nature, 619 (2023), pp. 533–538.

[8] P. Binev, A. Cohen, W. Dahmen, R. DeVore, G. Petrova, and P. Wojtaszczyk, *Data assimilation in reduced modeling*, SIAM/ASA Journal on Uncertainty Quantification, 5 (2017), pp. 1–29.

[9] B. Bonev, T. Kurth, C. Hundt, J. Pathak, M. Baust, K. Kashinath, and A. Anandkumar, *Spherical fourier neural operators: Learning stable dynamics on the sphere*, in International conference on machine learning, PMLR, 2023, pp. 2806–2823.

[10] L. Bottou and O. Bousquet, *The tradeoffs of large scale learning*, Advances in neural information processing systems, 20 (2007).

[11] N. Boullé and A. Townsend, *Learning elliptic partial differential equations with randomized linear algebra*, Foundations of Computational Mathematics, (2022), pp. 1–31.

[12] D. Braess and W. Hackbusch, *A new convergence proof for the multigrid method including the v-cycle*, SIAM journal on numerical analysis, 20 (1983), pp. 967–975.

[13] W. L. Briggs, V. E. Henson, and S. F. McCormick, *A multigrid tutorial*, SIAM, 2000.

[14] O. P. Bruno, Y. Han, and M. M. Pohlman, *Accurate, high-order representation of complex three-dimensional surfaces via Fourier continuation analysis*, Journal of computational Physics, 227 (2007), pp. 1094–1125.

[15] E. Calvello, N. B. Kovachki, M. E. Levine, and A. M. Stuart, *Continuum attention for neural operators*, Journal of Machine Learning Research, 26 (2025), pp. 1–52.

[16] L. Cao, T. O'Leary-Roseberry, P. K. Jha, J. T. Oden, and O. Ghattas, *Residual-based error correction for neural operator accelerated infinite-dimensional bayesian inverse problems*, Journal of Computational Physics, 486 (2023), p. 112104.

[17] S. Cao, *Choose a transformer: Fourier or galerkin*, Advances in neural information processing systems, 34 (2021), pp. 24924–24940.

[18] K. Carlberg, C. Farhat, J. Cortial, and D. Amsallem, *The gnat method for nonlinear model reduction: effective implementation and application to computational fluid dynamics and turbulent flows*, Journal of Computational Physics, 242 (2013), pp. 623–647.

[19] G. Chen, X. Liu, Q. Meng, L. Chen, C. Liu, and Y. Li, *Learning neural operators on riemannian manifolds*, National Science Open, 3 (2024), p. 20240001.

[20] J. Chen, K. Wu, and D. Xiu, *Due: A deep learning framework and library for modeling*

unknown equations, SIAM Review, 67 (2025), pp. 873–902.
[21] T. Chen and H. Chen, *Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems*, IEEE Transactions on Neural Networks, 6 (1995), pp. 911–917.
[22] Y. Chen, B. Hosseini, H. Owhadi, and A. M. Stuart, *Solving and learning nonlinear PDEs with Gaussian processes*, Journal of Computational Physics, 447 (2021), p. 110668.
[23] P. Cisneros-Velarde, B. Shrimali, and A. Banerjee, *Optimization for neural operators can benefit from width*, in Forty-second International Conference on Machine Learning, 2025, https://openreview.net/forum?id=mLysCaKFg7.
[24] E. Cleary, A. Garbuno-Inigo, S. Lan, T. Schneider, and A. M. Stuart, *Calibrate, emulate, sample*, Journal of Computational Physics, 424 (2021), p. 109716.
[25] A. Cohen, W. Dahmen, and R. DeVore, *State estimation—the role of reduced models*, in Recent Advances in Industrial and Applied Mathematics, T. Chacón Rebollo, R. Donat, and I. Higueras, eds., Cham, 2022, Springer International Publishing, pp. 57–77.
[26] N. C. Constantinou, G. L. Wagner, L. Siegelman, B. C. Pearson, and A. Palóczy, *Geophysicalflows.jl: Solvers for geophysical fluid dynamics problems in periodic domains on cpus & gpus*, Journal of Open Source Software, 6 (2021), p. 3053, https://doi.org/10.21105/joss.03053, https://doi.org/10.21105/joss.03053.
[27] S. M. Cox and P. C. Matthews, *Exponential time differencing for stiff systems*, Journal of Computational Physics, 176 (2002), pp. 430–455.
[28] M. V. de Hoop, N. B. Kovachki, N. H. Nelsen, and A. M. Stuart, *Convergence rates for learning linear operators from noisy data*, SIAM/ASA Journal on Uncertainty Quantification, 11 (2023), pp. 480–513.
[29] R. A. DeVore, *The theoretical foundation of reduced basis methods*, Model Reduction and approximation: Theory and Algorithms, (2014), pp. 137–168.
[30] W. E, *A proposal on machine learning via dynamical systems*, Communications in Mathematics and Statistics, 5 (2017), pp. 1–11, https://doi.org/10.1007/s40304-017-0103-z, https://journal.hep.com.cn/cimas/EN/10.1007/s40304-017-0103-z.
[31] M. Elrefaie, F. Morar, A. Dai, and F. Ahmed, *Drivaernet++: A large-scale multimodal car dataset with computational fluid dynamics simulations and deep learning benchmarks*, Advances in Neural Information Processing Systems, 37 (2024), pp. 499–536.
[32] C. Foias and R. Temam, *Gevrey class regularity for the solutions of the navier-stokes equations*, Journal of Functional Analysis, 87 (1989), pp. 359–369.
[33] Z. Gao, L. Yan, and T. Zhou, *Adaptive operator learning for infinite-dimensional bayesian inverse problems*, SIAM/ASA Journal on Uncertainty Quantification, 12 (2024), pp. 1389–1423.
[34] C. R. Gin, D. E. Shea, S. L. Brunton, and J. N. Kutz, *Deepgreen: deep learning of green's functions for nonlinear boundary value problems*, Scientific reports, 11 (2021), p. 21614.
[35] P. Gomes, M. Mara, J. Ho, K. Leppkes, G. Saez, R. Chiodi, T. D. Economon, and J. Alonso, *Combining and comparing aerodynamic shape optimization approaches on gpus: Adjoint methods and physics ai*, in AIAA SCITECH 2026 Forum, 2026, p. 2015.
[36] I. V. Gosea, S. Gugercin, and C. Beattie, *Data-driven balancing of linear dynamical systems*, SIAM Journal on Scientific Computing, 44 (2022), pp. A554–A582.
[37] S. Grimberg, C. Farhat, R. Tezaur, and C. Bou-Mosleh, *Mesh sampling and weighting for the hyperreduction of nonlinear petrov–galerkin reduced-order models with local reduced-order bases*, International Journal for Numerical Methods in Engineering, 122 (2021), pp. 1846–1874.
[38] G. Gupta, X. Xiao, and P. Bogdan, *Multiwavelet-based operator learning for differential equations*, Advances in neural information processing systems, 34 (2021), pp. 24048–24062.
[39] E. Haber and L. Ruthotto, *Stable architectures for deep neural networks*, Inverse problems, 34 (2018), p. 014004.
[40] E. Hairer, G. Wanner, and S. P. Nørsett, *Solving ordinary differential equations I: Non-stiff problems*, Springer, 1993.
[41] M. Han, D. Z. Huang, Y. Wang, Y. Zhang, and J. Zhou, *Geometric generalization of neural operators from kernel integral perspective*, arXiv preprint arXiv:2602.01498, (2026).
[42] W. Hao, R. P. Li, Y. Xi, T. Xu, and Y. Yang, *Multiscale neural networks for approximating green's functions*, SIAM Journal on Scientific Computing, 48 (2026), pp. C240–C270.
[43] M. Haywood-Alexander, G. Duthé, and E. Chatzi, *Piggo: Physics-guided learnable graph kalman filters for virtual sensing of nonlinear dynamic structures under uncertainty*, arXiv preprint arXiv:2604.26593, (2026).
[44] J. He, X. Liu, and J. Xu, *Mgno: Efficient parameterization of linear operators via multigrid*,

in International Conference on Learning Representations, vol. 2024, 2024, pp. 53409–53428.

[45] K. He, X. Zhang, S. Ren, and J. Sun, *Deep residual learning for image recognition*, in Proceedings of the IEEE conference on computer vision and pattern recognition, 2016, pp. 770–778.

[46] D. Hendrycks and K. Gimpel, *Gaussian error linear units (gelus)*, arXiv preprint arXiv:1606.08415, (2016).

[47] J. Hess and A. Smith, *Calculation of potential flow about arbitrary bodies*, Progress in aeronautical sciences, 8 (1967), pp. 1–138.

[48] J. S. Hesthaven, D. I. Gottlieb, and S. Gottlieb, *Spectral methods for time-dependent problems*, vol. 21, Cambridge University Press Cambridge, 2007.

[49] J. S. Hesthaven and S. Ubbiali, *Non-intrusive reduced order modeling of nonlinear problems using neural networks*, Journal of Computational Physics, 363 (2018), pp. 55–78.

[50] M. V. d. Hoop, D. Z. Huang, E. Qian, and A. M. Stuart, *The cost-accuracy trade-off in operator learning with neural networks*, Journal of Machine Learning, 1 (2022), pp. 299–341, https://doi.org/10.4208/jml.220509, https://doi.org/10.4208/jml.220509.

[51] J. Hu and P. Jin, *A hybrid iterative method based on mionet for PDEs: Theory and numerical examples*, Mathematics of Computation, (2025).

[52] Z. Hu, Q. Cao, K. Kawaguchi, and G. E. Karniadakis, *Deepomamba: State-space model for spatio-temporal pde neural operator learning*, Journal of Computational Physics, (2025), p. 114272.

[53] D. Huang, H. He, J. Harlim, and Y. Li, *Learning vector fields of differential equations on manifolds with geometrically constrained operator-valued kernels*, in International Conference on Learning Representations (ICLR), 2025.

[54] D. Z. Huang, N. H. Nelsen, and M. Trautner, *An operator learning perspective on parameter-to-observable maps*, Foundations of Data Science, 7 (2025), pp. 163–225, https://doi.org/10.3934/fods.2024037, https://www.aimsciences.org/article/id/66c30c8ae7a25d6c964d7718.

[55] P. Jin, Z. Zhang, A. Zhu, Y. Tang, and G. E. Karniadakis, *Sympnets: Intrinsic structure-preserving symplectic networks for identifying hamiltonian systems*, Neural Networks, 132 (2020), pp. 166–179.

[56] A. Jnini, E. Kiyani, K. Shukla, J. F. Urban, N. A. Daryakenari, J. Müller, M. Zeinhofer, and G. E. Karniadakis, *Curvature-aware optimization for high-accuracy physics-informed neural networks*, Computer Methods in Applied Mechanics and Engineering, 462 (2026), p. 119289.

[57] M. C. Kennedy and A. O'Hagan, *Bayesian calibration of computer models*, Journal of the Royal Statistical Society: Series B (Statistical Methodology), 63 (2001), pp. 425–464.

[58] Y. Khoo and L. Ying, *Switchnet: a neural network model for forward and inverse scattering problems*, SIAM Journal on Scientific Computing, 41 (2019), pp. A3182–A3201.

[59] T. Kim and M. Kang, *Bounding the rademacher complexity of Fourier neural operators*, Machine Learning, 113 (2024), pp. 2467–2498.

[60] E. Kiyani, K. Shukla, J. F. Urbán, J. Darbon, and G. E. Karniadakis, *Optimizing the optimizer for physics-informed neural networks and Kolmogorov-Arnold networks*, Computer Methods in Applied Mechanics and Engineering, 446 (2025), p. 118308.

[61] B. O. Koopman, *Hamiltonian systems and transformation in hilbert space*, Proceedings of the National Academy of Sciences, 17 (1931), pp. 315–318.

[62] J. Kossaifi, N. Kovachki, Z. Li, D. Pitt, M. Liu-Schiaffini, R. J. George, B. Bonev, K. Azizzadenesheli, J. Berner, V. Duruisseaux, and A. Anandkumar, *A library for learning neural operators*, Journal of Machine Learning Research, 27 (2026), pp. 1–6.

[63] N. Kovachki, S. Lanthaler, and S. Mishra, *On universal approximation and error bounds for Fourier neural operators*, Journal of Machine Learning Research, 22 (2021), pp. 1–76.

[64] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, *Neural operator: learning maps between function spaces with applications to PDEs*, Journal of Machine Learning Research, 24 (2023), pp. 1–97.

[65] R. Kress, *Linear integral equations*, 1989.

[66] T. Kurth, S. Subramanian, P. Harrington, J. Pathak, M. Mardani, D. Hall, A. Miele, K. Kashinath, and A. Anandkumar, *Fourcastnet: Accelerating global high-resolution weather forecasting using adaptive fourier neural operators*, in Proceedings of the platform for advanced scientific computing conference, 2023, pp. 1–11.

[67] S. Lanthaler, Z. Li, and A. M. Stuart, *Nonlocality and nonlinearity implies universality in operator learning*, Constructive Approximation, 62 (2025), pp. 261–303.

[68] S. Lanthaler, S. Mishra, and G. E. Karniadakis, *Error estimates for deeponets: A deep*

learning framework in infinite dimensions, Transactions of Mathematics and its Applications, 6 (2022), p. tnac001.

[69] J. Li, X. Du, and J. R. Martins, *Machine learning in aerodynamic shape optimization*, Progress in Aerospace Sciences, 134 (2022), p. 100849.

[70] Z. Li, D. Z. Huang, B. Liu, and A. Anandkumar, *Fourier neural operator with learned deformations for pdes on general geometries*, Journal of Machine Learning Research, 24 (2023), pp. 1–26, http://jmlr.org/papers/v24/23-0064.html.

[71] Z. Li, N. B. Kovachki, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, A. Anandkumar, et al., *Fourier neural operator for parametric partial differential equations*, in International Conference on Learning Representations, 2020.

[72] Z. Li, M. Liu-Schiaffini, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, *Learning chaotic dynamics in dissipative systems*, Advances in Neural Information Processing Systems, 35 (2022), pp. 16768–16781.

[73] L. E. Lingsch, M. Y. Michelis, E. de Bezenac, S. M. Perera, R. K. Katzschmann, and S. Mishra, *Beyond regular grids: Fourier-based neural operators on arbitrary domains*, in Forty-first International Conference on Machine Learning, 2024, https://openreview.net/forum?id=aVqqoFAavs.

[74] H. Liu, H. Yang, M. Chen, T. Zhao, and W. Liao, *Deep nonparametric estimation of operators between infinite dimensional spaces*, Journal of Machine Learning Research, 25 (2024), pp. 1–67.

[75] H. Liu, Z. Zhang, W. Liao, and H. Schaeffer, *Neural scaling laws of deep relu and deep operator network: A theoretical study*, arXiv preprint arXiv:2410.00357, (2024).

[76] Z. Liu, Y. Wu, D. Z. Huang, H. Zhang, X. Qian, and S. Song, *SPFNO: Spectral operator learning for PDEs with dirichlet and neumann boundary conditions*, arXiv preprint arXiv:2312.06980, (2023).

[77] M. Liu-Schiaffini, J. Berner, B. Bonev, T. Kurth, K. Azizzadenesheli, and A. Anandkumar, *Neural operators with localized integral and differential kernels*, in Forty-first International Conference on Machine Learning, 2024, https://openreview.net/forum?id=vl9GB3fbht.

[78] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, *Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators*, Nature Machine Intelligence, 3 (2021), pp. 218–229.

[79] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, and G. E. Karniadakis, *A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data*, Computer Methods in Applied Mechanics and Engineering, 393 (2022), p. 114778.

[80] L. Lu, X. Meng, Z. Mao, and G. E. Karniadakis, *Deepxde: A deep learning library for solving differential equations*, SIAM review, 63 (2021), pp. 208–228.

[81] D. Luo, T. O'Leary-Roseberry, P. Chen, and O. Ghattas, *Efficient PDE-constrained optimization under high-dimensional uncertainty using derivative-informed neural operators*, SIAM Journal on Scientific Computing, 47 (2025), pp. C899–C931.

[82] Y. Makara, Y. Tanaka, T. Matsubara, and T. Yaguchi, *Symplectic neural operators for learning infinite dimensional hamiltonian systems*, arXiv preprint arXiv:2605.15881, (2026).

[83] N. McGreivy and A. Hakim, *Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations*, Nature machine intelligence, 6 (2024), pp. 1256–1269.

[84] F. R. Menter, *Two-equation eddy-viscosity turbulence models for engineering applications*, AIAA journal, 32 (1994), pp. 1598–1605.

[85] F. R. Menter, M. Kuntz, R. Langtry, et al., *Ten years of industrial experience with the sst turbulence model*, Turbulence, heat and mass transfer, 4 (2003), pp. 625–632.

[86] I. Mezić, *Spectral properties of dynamical systems, model reduction and decompositions*, Nonlinear Dynamics, 41 (2005), pp. 309–325.

[87] I. Mezić, *Analysis of fluid flows via spectral properties of the Koopman operator*, Annual Review of Fluid Mechanics, 45 (2013), pp. 357–378.

[88] S. Mousavi, S. Wen, L. Lingsch, M. Herde, B. Raonic, and S. Mishra, *RIGNO: A graph-based framework for robust and accurate operator learning for PDEs on arbitrary domains*, in The Thirty-ninth Annual Conference on Neural Information Processing Systems, 2025, https://openreview.net/forum?id=ahJfROJOYt.

[89] N. H. Nelsen and A. M. Stuart, *The random feature model for input-output maps between banach spaces*, SIAM Journal on Scientific Computing, 43 (2021), pp. A3212–A3243.

[90] N. H. Nelsen and A. M. Stuart, *Operator learning using random features: A tool for*

scientific computing, SIAM Review, 66 (2024), pp. 535–571.

[91] OpenFOAM Foundation, *OpenFOAM User Guide: Mesh generation with* `snappyHexMesh`. https://doc.cfd.direct/openfoam/user-guide-v12/snappyhexmesh, 2024. Accessed 2026-04-22.

[92] S. A. Orszag and G. Patterson Jr, *Numerical simulation of three-dimensional homogeneous isotropic turbulence*, Physical Review Letters, 28 (1972), p. 76.

[93] S. V. Patankar and D. B. Spalding, *A calculation procedure for heat, mass and momentum transfer in three-dimensional parabolic flows*, in Numerical prediction of flow, heat transfer, turbulence and combustion, Elsevier, 1983, pp. 54–73.

[94] B. Peherstorfer and K. Willcox, *Data-driven operator inference for nonintrusive projection-based model reduction*, Computer Methods in Applied Mechanics and Engineering, 306 (2016), pp. 196–215.

[95] B. Peherstorfer, K. Willcox, and M. Gunzburger, *Survey of multifidelity methods in uncertainty propagation, inference, and optimization*, Siam Review, 60 (2018), pp. 550–591.

[96] E. Qian, B. Kramer, B. Peherstorfer, and K. Willcox, *Lift & learn: Physics-informed machine learning for large-scale nonlinear dynamical systems*, Physica D: Nonlinear Phenomena, 406 (2020), p. 132401.

[97] F. Rathgeber, D. A. Ham, L. Mitchell, M. Lange, F. Luporini, A. T. McRae, G.-T. Bercea, G. R. Markall, and P. H. Kelly, *Firedrake: automating the finite element method by composing abstractions*, ACM Transactions on Mathematical Software (TOMS), 43 (2016), pp. 1–27.

[98] N. Reinhardt, S. Wang, and J. Zech, *Statistical learning theory for neural operators*, Journal of Machine Learning Research, 27 (2026), pp. 1–82.

[99] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. Van Sloun, and Y. C. Eldar, *Kalmannet: Neural network aided kalman filtering for partially known dynamics*, IEEE Transactions on Signal Processing, 70 (2022), pp. 1532–1547.

[100] C. W. Rowley, I. Mezić, S. Bagheri, P. Schlatter, and D. S. Henningson, *Spectral analysis of nonlinear flows*, Journal of fluid mechanics, 641 (2009), pp. 115–127.

[101] P. J. Schmid, *Dynamic mode decomposition of numerical and experimental data*, Journal of fluid mechanics, 656 (2010), pp. 5–28.

[102] C. Schwab, A. Stein, and J. Zech, *Deep operator network approximation rates for lipschitz operators*, Analysis and Applications, 24 (2026), pp. 199–239.

[103] A. M. Stuart and M. Trautner, *Enforcing dirichlet boundary conditions in operator learning*, arXiv preprint arXiv:2608.27256, (2026).

[104] The OpenFOAM Foundation, *OpenFOAM 12*. https://openfoam.org/version/12/, 2024. Open-source CFD toolbox, accessed 2026-04-22.

[105] U. Trottenberg, C. W. Oosterlee, and A. Schuller, *Multigrid methods*, Academic press, 2001.

[106] E.-M. Walz, A. Henzi, J. Ziegel, and T. Gneiting, *Easy uncertainty quantification (easyuq): Generating predictive distributions from single-valued model output*, Siam Review, 66 (2024), pp. 91–122.

[107] A. Weihs, J. Sun, Z. Zhang, and H. Schaeffer, *A deep learning framework for multi-operator learning: Architectures and approximation theory*, arXiv preprint arXiv:2510.25379, (2025).

[108] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby, *A tensorial approach to computational continuum mechanics using object-oriented techniques*, Computers in physics, 12 (1998), pp. 620–631.

[109] S. Wiewel, M. Becher, and N. Thuerey, *Latent space physics: Towards learning the temporal evolution of fluid flow*, Computer Graphics Forum, 38 (2019), pp. 71–82, https://doi.org/10.1111/cgf.13620, https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.13620.

[110] N. Winovich, M. Daneker, L. Lu, and G. Lin, *Active operator learning with predictive uncertainty quantification for partial differential equations*, Journal of Computational Physics, (2026), p. 114791.

[111] H. Wu, H. Luo, H. Wang, J. Wang, and M. Long, *Transolver: A fast transformer solver for PDEs on general geometries*, in Forty-first International Conference on Machine Learning, 2024, https://openreview.net/forum?id=Ywl6pODXjB.

[112] J. Xu, *Iterative methods by space decomposition and subspace correction*, SIAM review, 34 (1992), pp. 581–613.

[113] J. Xu and L. Zikatanov, *The method of alternating projections and the method of subspace corrections in hilbert space*, Journal of the American Mathematical Society, 15 (2002), pp. 573–597.

[114] C. Zeng, Y. Zhang, J. Zhou, Y. Wang, Z. Wang, Y. Liu, L. Wu, and D. Z. Huang, *Point cloud neural operator for parametric PDEs on complex and variable geometries*, Computer Methods in Applied Mechanics and Engineering, 443 (2025), p. 118022.

[115] E. Zhang, A. Kahana, A. Kopaničáková, E. Turkel, R. Ranade, J. Pathak, and G. E. Karniadakis, *Blending neural operators and relaxation methods in PDE numerical solvers*, Nature Machine Intelligence, 6 (2024), pp. 1303–1313.

[116] H. Zhou and S. Cheng, *Improving long-term autoregressive spatiotemporal predictions: A proof of concept with fluid dynamics*, Computer Methods in Applied Mechanics and Engineering, 447 (2025), p. 118332.

[117] J. Zhou, V. Duruisseaux, D. Z. Huang, and A. Anandkumar, *Boundary-augmented neural operators for better generalization to unseen geometries*, in NeurIPS 2025 AI for Science Workshop, 2025.

[118] T. Zhou, X. Wan, D. Z. Huang, Z. Li, Z. Peng, A. Anandkumar, J. F. Brady, P. W. Sternberg, and C. Daraio, *Ai-aided geometric design of anti-infection catheters*, Science Advances, 10 (2024), p. eadj1741.

[119] Y. Zhu and N. Zabaras, *Bayesian deep convolutional encoder–decoder networks for surrogate modeling and uncertainty quantification*, Journal of Computational Physics, 366 (2018), pp. 415–447.

[120] Z. Zou, X. Meng, A. F. Psaros, and G. E. Karniadakis, *Neuraluq: A comprehensive library for uncertainty quantification in neural differential equations and operators*, SIAM Review, 66 (2024), pp. 161–190.